The Show Verse: plataforma web de gestión y descubrimiento de contenido audiovisual con integración de múltiples APIs

Trabajo de Fin de Grado

Convocatoria: Junio 2025

Alumno/a: Pablo Santos Conde

Tutor/a: Dr/Dra. D/Dña. Nombre y dos apellidos

Grado: Ingeniería del Software

Agradecimientos

Me gustaría agradecer a mi tutor de TFG por su orientación y disponibilidad a lo largo del proyecto. También a la comunidad de desarrolladores de las APIs de TMDb y Trakt, cuya documentación pública ha sido fundamental para el desarrollo de este trabajo.

Resumen

El presente Trabajo de Fin de Grado aborda el diseño y la implementación de *The Show Verse*, una aplicación web avanzada de gestión y descubrimiento de películas y series que permite a los usuarios unificar todo el contenido audiovisual en un solo lugar. La plataforma cuenta con secciones personalizadas para cada usuario, gestionadas a través de APIs externas, e integra funcionalidades de biblioteca local mediante Plex Media Server.

La dispersión del contenido entre las distintas plataformas de *streaming* genera una experiencia de usuario fragmentada y dificulta el seguimiento del contenido consumido. *The Show Verse* soluciona este problema ofreciendo una plataforma que unifica el contenido de las principales páginas de puntuaciones y plataformas de *streaming* haciendo uso de APIs, además de integrar el contenido local del usuario.

El trabajo consiste en el desarrollo completo de una aplicación web en Next.js que integra las APIs de The Movie Database (TMDb), Trakt.tv, OMDb, JustWatch y Plex Media Server. Se ha implementado un flujo de autenticación OAuth 2.0 completo con Trakt.tv y TMDb para permitir la sincronización del historial, los favoritos, la lista de pendientes, el calendario de estrenos y las valoraciones del usuario. Además, se ha diseñado un sistema de caché en múltiples niveles (ISR, memoria de proceso, sessionStorage, localStorage) con deduplicación en vuelo que reduce en un 70–80 % el número de peticiones a APIs externas en las vistas de listas.

La arquitectura implementada combina Server Components de React para la obtención inicial de datos con Client Components para la interactividad, logrando tiempos de carga (LCP) de 1,8 s y una puntuación de Performance en Lighthouse de 92/100. Se ha desarrollado un *middleware* Edge para optimización SEO con detección de bots y generación de metadatos Open Graph, un sistema de navegación responsive adaptativo (barra horizontal en desktop, drawer lateral + tab bar en móvil), y componentes con efectos visuales avanzados mediante Canvas 2D y Framer Motion.

Entre las funcionalidades destacables se incluyen: seguimiento episódico con visualización de progreso por temporada, grid de ratings que permite detectar patrones de calidad, integración con comentarios y listas de comunidad Trakt, sistema de artwork personalizado con sobrescrituras manuales, tres vistas alternativas (Grid, List, Compact) con persistencia de preferencias, actualización optimista de estado en todas las acciones de usuario, y gestión de errores con *graceful*

degradation.

Índice general

Resumen **2**

Índice de figuras **3**

Índice de tablas **4**

Abstract **5**

Glosario **6**

1 Introducción **7**
1.1 Justificación y contexto 7
1.2 Planteamiento del problema 8
1.3 Objetivos del trabajo 9

2 Estado de la cuestión **11**
2.1 Marco teórico del trabajo 11
2.2 Trabajos relacionados 13

3 Aspectos metodológicos **14**
3.1 Metodología 14
3.2 Tecnologías empleadas 17

4 Desarrollo del trabajo **18**
4.1 Arquitectura del sistema 18
4.2 Implementación del sistema de autenticación OAuth 2.0 20
4.3 Integración con las APIs externas 20
4.4 Implementación de los módulos funcionales 24
4.5 Diseño de la interfaz 44
4.6 Rendimiento en producción 51
4.7 Sistema de caché multinivel 51
4.8 Middleware y optimización SEO 55

4.9 Sistema de artwork personalizado 57
4.10 Integración con comunidad Trakt 59
4.11 Integración con Letterboxd 61
4.12 Manejo de errores y estados de carga 62
4.13 Diseño responsive y adaptación móvil 64

5 Conclusiones 66
5.1 Logros alcanzados 67
5.2 Discusión 68
5.3 Líneas de investigación futuras 69

Referencias 71

Anexo 72

Índice de figuras

Índice de tablas

3.1 Estrategia de caché por tipo de dato 16

4.1 Rutas principales de la aplicación 19
4.2 Principales funciones del cliente TMDb y sus *endpoints* 21
4.3 Principales *endpoints* de Trakt utilizados 22
4.4 Secciones del dashboard y sus fuentes de datos 24
4.5 Métricas Lighthouse en producción 51
4.6 Breakpoints responsive 64

5.1 Variables de entorno del proyecto 72
5.2 Sistema de caché multicapa 73

Abstract

This Final Degree Project presents the design and implementation of *The Show Verse*, a web application for the management and discovery of films and TV series that allows users to unify all audiovisual content in a single platform. The application integrates the TMDb, Trakt.tv and OMDb APIs for metadata and personal tracking, JustWatch for streaming availability, and Plex for access to content on a local media server. A complete OAuth 2.0 authentication flow with Trakt.tv has been implemented, together with a multi-level cache strategy that reduces external API calls by up to 80 % in list views. The result is a responsive web platform with fluid animations, three display modes (Grid, List and Compact), and Lighthouse scores of 92/100 in performance and 100/100 in SEO.

Glosario

API	*Application Programming Interface*. Interfaz de programación de aplicaciones.
CDN	*Content Delivery Network*. Red de distribución de contenidos.
CLS	*Cumulative Layout Shift*. Métrica de estabilidad visual de Core Web Vitals.
CSRF	*Cross-Site Request Forgery*. Tipo de ataque web que fuerza acciones no deseadas.
FID	*First Input Delay*. Tiempo de respuesta al primer input del usuario.
GraphQL	Lenguaje de consulta para APIs desarrollado por Meta.
httpOnly	Atributo de cookie que impide su acceso desde JavaScript del navegador.
ISR	*Incremental Static Regeneration*. Estrategia de caché de Next.js.
JWT	*JSON Web Token*. Estándar para la transmisión segura de información entre partes.
LCP	*Largest Contentful Paint*. Métrica de velocidad de carga de Core Web Vitals.
OAuth	*Open Authorization*. Protocolo estándar de delegación de acceso.
OMDb	*Open Movie Database*. Base de datos de películas con datos de IMDb.
REST	*Representational State Transfer*. Estilo arquitectónico para servicios web.
RSC	*React Server Components*. Componentes React que se ejecutan exclusivamente en servidor.
SPA	*Single Page Application*. Aplicación web de una sola página.
SPR	*Serverless Page Rendering*. Renderizado de páginas sin servidor propio.
SSR	*Server-Side Rendering*. Renderizado de páginas en el servidor.
TMDb	*The Movie Database*. Base de datos de películas y series con API pública.
TTL	*Time To Live*. Tiempo de vida de un elemento en caché.
XSS	*Cross-Site Scripting*. Tipo de ataque de inyección de código en el navegador.

Capítulo 1
Introducción

1.1 Justificación y contexto

En los últimos años, el consumo de películas y series ha cambiado radicalmente. Hoy en día la mayoría del contenido se distribuye a través de plataformas de *streaming* como Netflix, HBO Max, Disney+, Amazon Prime Video o Movistar+, cada una con su propio catálogo independiente, lo que genera un problema habitual para cualquier usuario: el contenido está disperso en múltiples servicios y no existe ninguna herramienta que permita gestionarlo todo desde un único lugar.

Esto provoca situaciones como perder el seguimiento de una serie que se dejó a medias, no recordar qué episodios se han visto, tener que buscar en varios sitios a la vez si una película está disponible en *streaming*, o simplemente no tener un registro de lo que ya se ha visto. Las plataformas de *tracking* que existen actualmente, como Trakt.tv o Letterboxd, resuelven parte del problema pero presentan sus propias limitaciones con falta de funcionalidades.

The Show Verse nace como respuesta a esa necesidad: una plataforma web que centraliza toda la gestión del contenido audiovisual, integrando los datos de múltiples fuentes mediante APIs externas y ofreciendo al usuario una experiencia completa y funcional.

Desde el punto de vista técnico, el proyecto aborda varios retos propios del desarrollo web actual: la integración de APIs REST de terceros con distintos esquemas de autenticación y formatos de datos, como la implementación del protocolo OAuth 2.0 para el acceso con Trakt.tv, el diseño de una estrategia de caché en múltiples niveles para optimizar el rendimiento, y la construcción de una interfaz responsiva con animaciones fluidas.

1.1.1 Motivación

El punto de partida de este trabajo es la experiencia personal como consumidor habitual de películas y series en múltiples plataformas de *streaming*. La dispersión del catálogo disponible entre los diferentes servicios provocaba con frecuencia situaciones de dificultad para recordar el historial

de películas y series y la puntuación que se le había dado a cada una, además de tener que buscar las puntuaciones y críticas en distintas webs independientes.

Las aplicaciones existentes en el mercado (Letterboxd, Trakt.tv, Simkl) ofrecen soluciones parciales, pero ninguna combina todos los datos y funcionalidades realmente necesarios a la hora de gestionar este tipo de contenido. Trakt.tv, la más completa en cuanto a API y funcionalidades de *tracking*, presenta una interfaz datada que resulta poco motivadora para el uso diario. Letterboxd, más elegante visualmente, carece de soporte completo para series de televisión y para otras funcionalidades importantes.

1.2 Planteamiento del problema

El problema abordado en este trabajo puede descomponerse en varias categorías técnicas.

1. Integración de fuentes de datos. Los datos necesarios para una plataforma de gestión de contenido audiovisual completa no están disponibles en una única API. TMDb proporciona los metadatos de películas y series (sinopsis, reparto, imágenes, trailers), además de las puntuaciones y listas de usuario. Trakt.tv gestiona el *tracking* personal del usuario (historial de visionado, sección en progreso y estadísticas). OMDb complementa con las puntuaciones de IMDb y Rotten Tomatoes. Cada API tiene su propio esquema de autenticación, formato de respuesta, límites de uso e identificadores.

2. Autenticación y gestión de sesiones. La funcionalidad de sincronización con Trakt requiere que el usuario delegue permisos en la aplicación mediante OAuth 2.0, lo que implica implementar el flujo completo de autorización (redirección, intercambio de código por token, almacenamiento seguro, refresco automático), sin exponer en ningún momento las credenciales del usuario ni las claves privadas de la aplicación.

3. Rendimiento y experiencia de usuario. Este tipo de aplicación maneja grandes volúmenes de imágenes de alta resolución y realiza múltiples llamadas a APIs externas con límites de peticiones en algunos casos reducidos. Es importante que haya un equilibrio entre datos almacenados en caché y aquellos que se muestran en tiempo real.

4. Sincronización de estado entre servidor y cliente. Las acciones del usuario (marcar como visto, añadir a favoritos, puntuar) deben reflejarse de forma inmediata en la interfaz sin esperar la confirmación del servidor, pero manteniendo la coherencia en caso de error.

1.3 Objetivos del trabajo

Objetivo general: diseñar e implementar una aplicación web completa y funcional para la gestión personalizada y el descubrimiento de nuevas películas y series, que integre múltiples APIs externas y ofrezca una experiencia de usuario centralizada, moderna y eficiente.

Objetivos específicos:

1) Implementar la integración con la API de TMDb para obtener los metadatos completos de películas y series (sinopsis, características, imágenes, vídeos, temporadas y episodios).

2) Desarrollar un sistema de autenticación OAuth 2.0 completo con Trakt.tv, incluyendo almacenamiento seguro de tokens en cookies `httpOnly`, actualización automática mediante *refresh tokens* y revocación.

3) Construir un módulo de gestión personal (favoritos, pendientes, historial de visionado, series en progreso y puntuaciones de usuario) con sincronización bidireccional con Trakt y TMDb.

4) Implementar un sistema de seguimiento episódico completo para series, incluyendo marcado individual por episodio, marcado masivo por temporada, visualización de progreso y grid de ratings para detección de patrones de calidad.

5) Diseñar una interfaz de usuario responsiva y accesible que ofrezca múltiples modos de visualización (Grid, List, Compact) con transiciones fluidas mediante Framer Motion y compatibilidad con dispositivos móviles mediante navegación adaptativa.

6) Optimizar el rendimiento de la aplicación mediante un sistema de caché multinivel (ISR en servidor, memoria de proceso Node.js, sessionStorage, localStorage) con deduplicación en vuelo para reducir el tráfico API.

7) Implementar un *middleware* Edge para optimización SEO con detección de bots, generación de metadatos Open Graph y reescritura de URLs para garantizar indexación correcta.

8) Integrar Plex Media Server para acceso a biblioteca local del usuario, incluyendo autenticación JWT, enlaces directos multiplataforma y búsqueda unificada.

9) Desarrollar componentes interactivos avanzados con efectos visuales mediante Canvas 2D (LiquidButton) y sistema de actualización optimista para todas las acciones de usuario.

10) Implementar gestión de errores con *graceful degradation* y *skeleton loaders* para garantizar funcionalidad parcial aunque fallen APIs externas.

11) Desplegar la aplicación en un entorno de producción (Vercel) con métricas de rendimiento óptimas (LCP <2 s, CLS <0,1, Performance >90/100).

Capítulo 2
Estado de la cuestión

El presente estado de la cuestión aborda el contexto tecnológico y de mercado en el que se enmarca el proyecto. Se comienza con una visión del sector de las plataformas de *streaming* y sus implicaciones para el usuario; a continuación se analizan las tecnologías de desarrollo web actuales que hacen posible la solución propuesta; y se finaliza examinando las plataformas existentes más relevantes.

2.1 Marco teórico del trabajo

2.1.1 Las plataformas de streaming y sus consecuencias para el usuario

La transición del modelo tradicional al *streaming* bajo demanda se inició con el lanzamiento del servicio de Netflix en 2007, aunque su consolidación definitiva se produjo a partir de 2019, con la irrupción simultánea de Disney+, Apple TV+ y HBO Max. En 2024, el usuario medio en mercados como España o Estados Unidos tiene acceso simultáneo a entre 3 y 5 servicios de *streaming* distintos, según datos de CNMC y Statista (CNMC, 2024; Statista, 2024).

Esta fragmentación genera una sobrecarga de elección en cada usuario, lo que provoca que inviertan una gran cantidad de tiempo en decidir qué ver, olviden mantener un registro de lo visto y pierdan el seguimiento de las series que consumen de forma no continua (Schwartz, 2004).

La necesidad de una capa de abstracción sobre las plataformas de *streaming* ha generado el desarrollo de aplicaciones conocidas como **media trackers**.

2.1.2 Protocolos de autenticación: OAuth 2.0

OAuth 2.0 es el estándar que permite a una aplicación actuar en nombre de un usuario en un servicio externo sin que ese servicio tenga que proporcionarle su contraseña (Hardt, 2012). En *The Show Verse* se utiliza para conectar la cuenta de Trakt.tv del usuario con la aplicación.

El flujo implementado (Authorization Code) funciona de la siguiente manera:

1) Cuando el usuario pulsa «Conectar con Trakt», la ruta `/api/trakt/auth/start` genera un código aleatorio de seguridad (`state`) para evitar ataques CSRF, construye la URL de autorización de Trakt y redirige al usuario.

2) El usuario introduce sus credenciales en Trakt y concede permisos. Trakt redirige de vuelta a `/api/trakt/auth/callback` incluyendo un `code` de un solo uso y el mismo `state`.

3) La ruta de *callback* verifica el `state` e intercambia el `code` por un `access_token` y un `refresh_token` directamente desde el servidor.

4) Ambos tokens se guardan en cookies `httpOnly` con los atributos `Secure` y `SameSite=Lax`, de forma que el JavaScript del navegador no puede leerlos en ningún momento.

5) Cuando el `access_token` expira, la aplicación lo renueva automáticamente usando el `refresh_token`.

2.1.3 APIs REST

Las APIs utilizadas en este proyecto siguen el estilo REST (Fielding, 2000) con las siguientes especificidades:

TMDb API v3 es una API REST pública con autenticación por API Key. Proporciona acceso a su base de datos de más de 800.000 películas y 150.000 series de televisión, con soporte para más de 40 idiomas. Su límite de peticiones es de 40 peticiones por 10 segundos para cuentas gratuitas (The Movie Database, 2024).

Trakt.tv API v2 combina autenticación por API Key para *endpoints* públicos con OAuth 2.0 Bearer Token para *endpoints* autenticados. Un aspecto técnico relevante es el sistema de identificadores: Trakt mantiene sus propios slugs e IDs, pero acepta identificadores de IMDb y TMDb mediante el *endpoint* de búsqueda por ID externo (Trakt, 2024).

2.1.4 Rendimiento web

Las estrategias de optimización más relevantes para este proyecto incluyen: Server-Side Rendering para reducir el LCP, el componente `<Image>` de Next.js para optimización automática de imágenes (conversión a WebP, *lazy loading*, reserva de espacio para prevenir CLS), y el *code splitting* automático por ruta para reducir el tiempo de carga inicial (Google Developers, 2024a, 2024b).

2.1.5 Diseño de interfaz

El diseño de *The Show Verse* se basa en el uso de los *backdrops* y *posters* obtenidos de TMDb como elemento principal de la interfaz.

Modo oscuro y backdrops de fondo. El color de base de la aplicación es el negro (`#000000`). Las imágenes de fondo se despliegan a pantalla completa con un gradiente en los bordes. Los elementos superpuestos utilizan un fondo semi-transparente con desenfoque del contenido detrás (`backdrop-filter: blur`), siguiendo el patrón de *glassmorphism* (Babich, 2021).

Animaciones de entrada y transiciones. Las animaciones se gestionan mediante Framer Motion (Framer, 2024), usando `AnimatePresence` para la entrada y salida de componentes. Para listas largas (más de 30 elementos), el retraso entre tarjetas se reduce automáticamente para que la animación no resulte lenta.

2.2 Trabajos relacionados

Letterboxd (Letterboxd, 2024) (fundado en 2011, Auckland, Nueva Zelanda) es la plataforma de referencia para el registro y la crítica cinematográfica personal. Con más de 20 millones de usuarios registrados en 2024, su principal fortaleza es la componente social y el sistema de reseñas. Su limitación más significativa para los propósitos de este trabajo es la ausencia de soporte para el seguimiento de series de televisión por episodio.

Trakt.tv (fundado en 2010) es la plataforma más completa en cuanto a funcionalidades de *tracking*, con soporte completo para películas y series a nivel de episodio. Su característica diferencial es la API pública v2, documentada y con límites de uso generosos para desarrolladores registrados. Sin embargo, su interfaz de usuario presenta un diseño anticuado que no ha evolucionado significativamente en la última década.

Simkl (fundado en 2012) ofrece funcionalidades similares a Trakt con una interfaz algo más moderna, pero con un ecosistema de API menos estable y una comunidad de usuarios significativamente menor.

JustWatch (fundado en 2014) se especializa en la localización de contenido entre plataformas de *streaming*, pero carece de funcionalidades de *tracking* personal.

La comparación entre estas plataformas pone de manifiesto una oportunidad en el mercado: ninguna combina la funcionalidad de Trakt (seguimiento por episodio, sincronización, valoraciones) con una interfaz de usuario moderna y responsiva comparable a las propias plataformas de *streaming*. *The Show Verse* se posiciona precisamente en este punto.

Capítulo 3
Aspectos metodológicos

El desarrollo de *The Show Verse* se ha abordado mediante una **metodología iterativa basada en el ciclo de vida ágil**, adaptada a un proyecto individual. La elección de este enfoque frente a las metodologías en cascada se justifica por la naturaleza exploratoria del proyecto: la integración con APIs de terceros implica incertidumbres técnicas que no siempre se pueden anticipar en una fase de análisis, y la prioridad entre funcionalidades puede variar a medida que se comprende mejor el problema.

El proceso de desarrollo se articula en iteraciones de aproximadamente una semana de duración. Al inicio de cada iteración se seleccionan del *backlog* las funcionalidades a abordar, priorizadas según tres criterios:

- **Dependencia técnica:** las funcionalidades que sirven de base a otras (autenticación, cliente de API) se abordan primero.
- **Valor aportado al usuario:** se priorizan las funcionalidades más visibles y relevantes para la experiencia de uso.
- **Riesgo técnico:** las integraciones más inciertas se adelantan para conocer cuanto antes sus restricciones.

3.1 Metodología

3.1.1 Criterios de integración de APIs

La selección de las APIs externas ha seguido los siguientes criterios:

- **Datos completos:** TMDb fue seleccionada frente a otras alternativas (The Open Movie Database, Wikidata) por ofrecer la mayor cobertura de contenido, soporte en diferentes idiomas, galería de imágenes y datos de *streaming*, además de disponer de la documentación más completa y una comunidad de desarrolladores activa.

- **Capacidad de tracking:** Trakt.tv fue seleccionada como plataforma de sincronización por su API pública v2 bien documentada, su soporte completo de series a nivel de episodio y por ser la plataforma de *tracking* más referenciada por la comunidad de desarrolladores independientes.
- **Complementariedad:** OMDb fue seleccionada únicamente para los datos de *rating* de IMDb y Rotten Tomatoes, que TMDb no proporciona directamente.

3.1.2 Criterios de decisión para la autenticación

El problema de la autenticación OAuth 2.0 admitía varias opciones de implementación en Next.js. Se descartó el uso de librerías de autenticación genéricas como NextAuth.js / Auth.js por las siguientes razones:

- Trakt.tv no es un proveedor OAuth soportado de forma nativa por estas librerías.
- La personalización necesaria (refresco de tokens específico de Trakt, intercambio de códigos con *headers* personalizados) habría requerido más configuración que implementar el flujo directamente.
- La implementación propia ofrece mayor control sobre el manejo de errores y el almacenamiento de tokens.

Se optó por implementar el flujo Authorization Code de forma manual mediante **API Routes de Next.js**, almacenando los tokens en cookies `httpOnly` con atributos `Secure` y `SameSite=Lax`. Esta decisión garantiza que los tokens no sean accesibles desde JavaScript del navegador (protección frente a XSS) y que no se envíen en peticiones *cross-site* no intencionadas (protección parcial frente a CSRF).

3.1.3 Estrategia de caché

Para cada tipo de dato gestionado en la aplicación se ha definido una estrategia de caché en función de la frecuencia de actualización del dato en origen y su importancia para la experiencia de usuario.

Esta diferenciación permite reducir el número de llamadas a APIs externas en un estimado del 70–80 % para los datos de catálogo, mientras se garantiza la actualización inmediata de los datos de usuario que requieren frescura absoluta.

Caché en múltiples niveles: el sistema implementa cuatro niveles de caché complementarios:

1) **ISR (Incremental Static Regeneration):** páginas estáticas regeneradas periódicamente en servidor, con servicio desde caché entre regeneraciones.

Tipo de dato	Almacenamiento	TTL
Catálogo general (top rated, populares)	ISR servidor	10 minutos
Detalles de película/serie	ISR servidor	1 hora
Imágenes TMDb	CDN Vercel	1 hora
Datos OMDb (proxy servidor)	Map en proceso	24 h / 60 s
Datos OMDb en cliente	sessionStorage	24 horas
Puntuaciones IMDb y Trakt en listas	localStorage	30 días
Rating personal del usuario	Map en módulo	10 min / 45 s
Estado personal del usuario	No-cache	En cada petición

Tabla 3.1: Estrategia de caché por tipo de dato

2) **Memory cache en proceso Node.js:** `Map` global que persiste durante la vida del proceso serverless (típicamente 5–15 min en Vercel), evitando peticiones redundantes a OMDb.

3) **sessionStorage en cliente:** caché de puntuaciones que persiste durante la sesión del navegador (hasta cerrar pestaña), común entre páginas.

4) **localStorage en cliente:** caché persistente de puntuaciones con TTL de 30 días, sobrevive entre sesiones del navegador.

Deduplicación en vuelo: implementada en `lib/api/omdb.js`, si múltiples componentes solicitan el mismo dato simultáneamente, solo se realiza una petición HTTP y todas las solicitudes esperan el resultado compartido. Esto es crítico en listas de 100+ ítems donde varias tarjetas pueden necesitar el mismo `imdbId`.

Pool de concurrencia: para respetar los *rate limits* de las APIs (OMDb: 1000 peticiones/día en plan gratuito), las peticiones paralelas se limitan a 4–5 simultáneas mediante un patrón de *worker pool*. Cuando hay 100 peticiones pendientes, se ejecutan en lotes de 4, esperando a que una termine antes de lanzar la siguiente.

Algoritmo LRU para localStorage: al superar 500 entradas cacheadas, se aplica *Least Recently Used* para eliminar entradas antiguas y mantener el tamaño del almacenamiento controlado (típicamente <2 MB).

3.2 Tecnologías empleadas

Next.js 16 (Framework principal). Seleccionado frente a otras alternativas (Remix, SvelteKit, Nuxt 3) por ser el framework React con mayor adopción industrial (>40 % de cuota entre frameworks React según npm trends 2024), la mejor integración con Vercel y la documentación más extensa. El App Router de Next.js 13+ es la única opción que implementa React Server Components de forma estable y con soporte oficial (Vercel, 2024).

React 19. La elección de React como biblioteca de UI es consecuencia directa de la elección de Next.js. React 19 introduce mejoras en el manejo de acciones de servidor y la promesa nativa en componentes de cliente (Meta Open Source, 2024).

Tailwind CSS 4. Seleccionado frente a CSS Modules, Styled Components o Emotion por el paradigma *utility-first* que acelera el desarrollo sin sacrificar la personalización. La versión 4, basada en PostCSS con motor Rust nativo, elimina el PurgeCSS separado e incorpora mejoras sustanciales de rendimiento en el tiempo de compilación.

Framer Motion 12. La elección de Framer Motion frente a otras librerías de animación (React Spring, CSS Animations puras) se basa en su modelo declarativo basado en variantes, su soporte nativo para animaciones de *layout* y su integración con el ciclo de vida de los componentes React mediante `AnimatePresence` (Framer, 2024).

TypeScript 5. Configurado en modo de verificación estricta para la configuración del proyecto y los tipos de datos de las respuestas de API.

Capítulo 4
Desarrollo del trabajo

En este capítulo se describe el desarrollo completo de *The Show Verse*, desde la arquitectura general del sistema hasta la implementación de cada módulo funcional y la integración con las APIs externas. Es la parte más importante del TFG y recoge las decisiones técnicas tomadas durante el proceso de construcción de la plataforma.

4.1 Arquitectura del sistema

4.1.1 Estructura de capas

El sistema se estructura en tres capas diferenciadas, implementadas en una única base de código Next.js:

Capa de presentación (Client Components). Componentes React que se ejecutan y se hidratan en el navegador. Gestionan el estado local de la interfaz, las interacciones del usuario y las peticiones a la capa intermedia. Los componentes de mayor complejidad son `MainDashboardClient.jsx` (gestión del dashboard completo), `DetailsClient.jsx` (página de detalle de película y serie) y `DiscoverClient.jsx` (módulo de búsqueda).

Capa de servidor (Server Components + API Routes). Los Server Components de Next.js realizan la búsqueda inicial de datos directamente en el servidor, sin coste de JavaScript para el cliente. Las API Routes actúan como *proxy* hacia las APIs externas, añadiendo los tokens de autenticación en el servidor y transformando los datos al formato que consume el *frontend*.

Capa de datos (APIs externas). TMDb, Trakt.tv, OMDb, JustWatch y Plex como fuentes para los datos de catálogo y del usuario respectivamente.

La separación entre estas capas se hace explícita en la convención de nombres del proyecto: los archivos terminados en `Client.jsx` son siempre Client Components (marcados con `"use client"`), mientras que los archivos `page.jsx` y los de la carpeta `api/` son siempre Server Components o API Routes.

4.1.2 Estructura de carpetas

El proyecto sigue la convención del App Router de Next.js 13+. La estructura principal se organiza en cuatro directorios bajo `src/`:

- `src/app/` — Rutas, páginas y API Routes. Cada subdirectorio corresponde a una ruta de la aplicación.
- `src/components/` — Componentes React reutilizables. Los de mayor peso son `DetailsClient.jsx` (∼331 KB) y `MainDashboardClient.jsx` (∼102 KB).
- `src/lib/` — Lógica de negocio, clientes de API y utilidades. Incluye los módulos `tmdb.js`, `traktClient.js`, `justwatch.js` y `plex/auth.js`.
- `src/context/` — Contextos globales de React para el estado compartido entre componentes.

Las rutas principales de la aplicación son:

Ruta	**Descripción**
`/`	Dashboard principal con carrusel y secciones
`/details/[type]/[id]`	Página de detalle de película o serie
`/favorites`	Lista de favoritos del usuario (TMDb)
`/watchlist`	Lista de pendientes del usuario (TMDb)
`/history`	Historial de visionado (Trakt)
`/in-progress`	Series en progreso (Trakt)
`/stats`	Estadísticas del usuario (Trakt)
`/calendar`	Calendario de estrenos y próximos episodios
`/discover`	Búsqueda y filtrado avanzado de contenido
`/biblioteca`	Librería personal unificada
`/lists`	Listas de TMDb, colecciones y listas de Trakt
`/movies`	Exploración de catálogo de películas
`/series`	Exploración de catálogo de series
`/details/person/[id]`	Ficha de actor o directora

Tabla 4.1: Rutas principales de la aplicación

4.2 Implementación del sistema de autenticación OAuth 2.0

La implementación del flujo OAuth 2.0 con Trakt.tv es el núcleo técnico más delicado del proyecto. El flujo se articula en cuatro API Routes de Next.js:

Inicio del flujo (`/api/trakt/oauth`): construye la URL de autorización de Trakt con los parámetros `client_id`, `redirect_uri` y `response_type=code`, y redirige al usuario. Este *endpoint* se invoca al pulsar el botón «Conectar con Trakt» de la Navbar.

Callback (`/api/trakt/auth/callback`): recibe el `authorization_code` de Trakt, lo intercambia por `access_token` y `refresh_token` mediante una petición POST servidor-a-servidor a `https://api.trakt.tv/oauth/token`, y almacena ambos tokens en cookies `httpOnly`. A continuación redirige al usuario a la página de inicio.

Estado (`/api/trakt/auth/status`): lee las cookies de sesión y, si existe `access_token` válido, hace una petición a `/users/me` de Trakt para obtener el perfil del usuario. Si el token está expirado, intenta refrescarlo usando el `refresh_token` antes de declarar al usuario como no autenticado.

Desconexión (`/api/trakt/auth/disconnect`): revoca el token en Trakt mediante la API `/oauth/revoke` y elimina las cookies de sesión.

Este diseño garantiza que ni el `client_secret` de Trakt ni los `access_token` del usuario sean accesibles desde el JavaScript del navegador en ningún momento del flujo.

4.3 Integración con las APIs externas

4.3.1 The Movie Database (TMDb) API v3

El cliente de TMDb (`lib/api/tmdb.js`) encapsula toda la lógica de acceso a la API v3. La función central `tmdb(path, params, options)` implementa:

- Un **constructor de URL unificado** que añade automáticamente la API Key y el idioma por defecto (`es-ES`).
- Un **mecanismo de timeout** mediante `AbortController` (8 segundos por defecto) para evitar que peticiones lentas bloqueen la interfaz.
- **Comportamiento de caché diferenciado**: `force-cache` con revalidación ISR en servidor, `no-store` en cliente.

- **Gestión granular de errores**: los errores 404 retornan `null` silenciosamente; otros errores se loguean y también retornan `null`.

Sobre esta función base se construyen más de 40 funciones nombradas por dominio (`fetchTopRatedMovies`, `getDetails`, `getCredits`, `getWatchProviders`, `getActorDetails`, etc.) que constituyen la API pública del módulo.

Un aspecto técnico importante es la resolución de imágenes: la función `getLogos` obtiene el logo oficial de un título aplicando un criterio de selección que prioriza logos en español, luego en inglés, luego sin idioma, seleccionando el de mayor número de votos dentro de cada grupo.

La Tabla 4.2 recoge los *endpoints* principales utilizados.

Función	**Endpoint TMDb**
`fetchTopRatedMovies()`	`GET /movie/top_rated`
`fetchTrendingMovies()`	`GET /trending/movie/week`
`getDetails(type, id)`	`GET /{type}/{id}?append_to_response=external`
`getLogos(type, id)`	`GET /{type}/{id}/images`
`getCredits(type, id)`	`GET /{type}/{id}/credits`
`getWatchProviders(type, id)`	`GET /{type}/{id}/watch/providers`
`getRecommendations(type, id)`	`GET /{type}/{id}/recommendations`
`markAsFavorite({...})`	`POST /account/{id}/favorite`
`markInWatchlist({...})`	`POST /account/{id}/watchlist`
`fetchFavoritesForUser(...)`	`GET /account/{id}/favorite/movies|tv`
`fetchWatchlistForUser(...)`	`GET /account/{id}/watchlist/movies|tv`

Tabla 4.2: Principales funciones del cliente TMDb y sus *endpoints*

4.3.2 OMDb API (Open Movie Database)

OMDb es la única fuente pública que permite obtener el *rating* de IMDb, el número de votos y las puntuaciones de Rotten Tomatoes y Metacritic por `imdbId`. Para proteger la API Key, todas las peticiones a OMDb pasan por la API Route interna `/api/omdb`.

El *proxy* implementa un **sistema de deduplicación** (`inflight Map` global): si múltiples tarjetas solicitan simultáneamente el mismo `imdbId`, solo se realiza una petición real a OMDb. Las respuestas exitosas se cachean 24 horas en el proceso de Node.js; las fallidas, 60 segundos

para reintentar pronto. La cabecera `Cache-Control: public, s-maxage=86400` permite adicionalmente que la CDN de Vercel cachee las respuestas.

4.3.3 Proxy de Trakt y transformación de datos

Las API Routes de Trakt actúan como intermediario que resuelve dos problemas estructurales:

Problema de identificadores. El *frontend* trabaja exclusivamente con `tmdbId`, mientras que muchos *endpoints* de Trakt requieren sus propios identificadores (`traktId`, `slug`). Las API Routes realizan internamente la traducción mediante el *endpoint* de búsqueda por ID externo de Trakt (`/search/tmdb/{id}?type=movie|show`).

Problema de enriquecimiento de datos. Las respuestas de Trakt incluyen datos de *tracking* pero no imágenes. Las API Routes que devuelven listas de contenido (historial, favoritos, pendientes) realizan peticiones paralelas a TMDb para obtener las URLs de *poster* y *backdrop*, enriqueciendo la respuesta antes de enviarla al cliente.

La tabla 4.3 recoge los *endpoints* de Trakt utilizados por módulo funcional.

Endpoint Trakt	**Descripción**
`GET /sync/history`	Historial completo de películas y episodios vistos
`POST /sync/history`	Registrar película/episodio como visto
`DELETE /sync/history`	Desmarcar como visto
`GET /sync/watched/shows`	Estado de visionado de todas las series
`GET /sync/watchlist`	Lista de pendientes del usuario
`POST /sync/ratings`	Añadir valoración (1–10)
`GET /users/me/stats`	Estadísticas globales del usuario
`GET /calendars/my/shows`	Próximos episodios de series seguidas
`GET /search/tmdb/{id}`	Búsqueda de item por ID de TMDb

Tabla 4.3: Principales *endpoints* de Trakt utilizados

4.3.4 JustWatch (API GraphQL no oficial)

JustWatch no tiene una API oficial pública. El cliente (`lib/api/justwatch.js`) utiliza los mismos *endpoints* GraphQL que la web de JustWatch, enviando peticiones `POST` a `https://apis.justwatch.com/graphql`.

Se utilizan dos *queries* GraphQL:

1) **Búsqueda del título** (`GetSearchTitles`): recibe el título y tipo de contenido y devuelve el `nodeId` interno de JustWatch.

2) **Detalles con ofertas** (`GetTitleDetails`): recibe el `nodeId` y devuelve las ofertas de *streaming* disponibles para España, incluyendo el tipo de monetización (`FLATRATE`, `RENT`, `BUY`), la URL directa al contenido en la plataforma y los datos del proveedor.

La función principal `getStreamingProviders(title, type, year)` filtra las ofertas de tipo `FLATRATE` (suscripción), les asigna el logo de la plataforma mediante un mapeo estático a los logos de TMDb y devuelve la URL directa al contenido.

4.3.5 Plex Media Server

La integración con Plex permite a los usuarios que tienen una biblioteca local abrir directamente cualquier película o serie desde *The Show Verse* en su servidor Plex, sin tener que buscar el contenido manualmente.

Autenticación. El módulo `lib/plex/auth.js` implementa dos modos:

- **Modo simple:** se configura `PLEX_TOKEN` en variables de entorno y se usa directamente en cada petición.
- **Modo JWT:** se configura una clave privada RSA o Ed25519 (`PLEX_JWT_PRIVATE_KEY`). La aplicación firma JWTs con la clave privada, los intercambia por tokens de acceso en la API de Plex y los renueva automáticamente antes de su expiración. La deduplicación de refrescos concurrentes se gestiona mediante un `Promise` compartido (`state.refreshPromise`).

API Route interna (`GET /api/plex`). Recibe `title`, `type`, `year`, `imdbId` y `tmdbId` como parámetros y devuelve varios formatos de URL para el acceso al contenido desde diferentes plataformas:

- `plexWebUrl`: URL para la interfaz web de Plex en escritorio.
- `plexUniversalUrl`: `https://watch.plex.tv/{type}/{slug}`, un enlace universal que abre la app nativa en iOS y Android.
- `plexSlugUrl`: `plex://{type}/{slug}`, *deep link* nativo de la aplicación.
- `plexAndroidSlugIntentUrl`: formato `intent://` para Chrome en Android.

El slug para las URLs universales se resuelve consultando `https://metadata.provider.plex.tv/li`

4.4 Implementación de los módulos funcionales

En esta sección se documentan en profundidad los módulos principales de la aplicación, explicando su arquitectura, flujos de datos, patrones de optimización y características funcionales.

4.4.1 Dashboard principal (/)

Es el punto de entrada de la aplicación y muestra un conjunto de secciones filtradas y organizadas por temas.

La página de inicio (`src/app/page.jsx`) es un Server Component que ejecuta en paralelo todas las peticiones necesarias de TMDb mediante `Promise.all()` lo que permite reducir el tiempo total de carga a la latencia de la petición más lenta.

A continuación se muestran las secciones principales del dashboard, su respectivo endpoint de TMDb y el resultado obtenido:

Sección	**Endpoint TMDb**	**Descripción**
Hero	`/trending/movie/week`	Carrusel de películas de este mes
Top Películas	`/movie/top_rated`	Películas mejor valoradas de todos los tiempos
Tendencias	`/trending/movie/week`	Películas populares esta semana
Populares	`/movie/popular`	Contenido con mayor momentum
Acción	`/discover/movie?with_genres=28`	Películas de acción clásicas
Dramas	`/discover/movie?with_genres=18`	Películas dramáticas imprescindibles
Series Populares	`/tv/popular`	Series trending del momento
Series Top Rated	`/tv/top_rated`	Series clásicas mejor valoradas
Series Trending	`/trending/tv/week`	Series en tendencia esta semana
Emitiéndose Hoy	`/tv/airing_today`	Episodios con emisión real-time
En Emisión	`/tv/on_the_air`	Series con temporada activa
Favoritos (auth)	`/account/{id}/favorite/movies`	Favoritos personales del usuario

Tabla 4.4: Secciones del dashboard y sus fuentes de datos

Selección de backdrops: el carrusel principal utiliza *backdrops* de TMDb y se aplica un criterio de selección a partir de la función `pickBestBackdropByLangResVotes(...)` que prioriza primero los backdrops en inglés (`en`, `en-US`) cuando existen, después filtra por resolución y por `vote_count` y `vote_average`.

4.4.2 Página de detalles (`/details/[type]/[id]`)

La página de detalles es el componente más complejo de la aplicación. Su funcionalidad es mostrar la información completa de la película o serie seleccionada, integrando datos de TMDb, Trakt.tv, OMDb (IMDb), Plex Media Server, JustWatch, Letterboxd y múltiples servicios adicionales. Además, permite al usuario interactuar con el contenido (marcar como favorito, añadir a pendientes, valorar, compartir, etc.) y navegar por secciones relacionadas (reparto, recomendaciones, colecciones).

Arquitectura de datos

Se hace uso de Server Component y Client Component que optimiza tanto el SEO como la interacción en la página web. El Server Component se encarga de obtener los datos críticos iniciales en tiempo de renderizado *server-side*, incluyendo los metadatos completos del título, los logos oficiales, el reparto y equipo técnico completo, las recomendaciones de contenido similar, los proveedores de *streaming* disponibles por país, los videos oficiales (trailers, teasers, clips) y las reseñas de la comunidad. Si el usuario está autenticado, también se obtiene el estado de la cuenta (si ha marcado el título como favorito, si está en su lista de pendientes, y su valoración personal).

Una vez que el Client Component se ejecuta en el navegador, se empiezan a realizar peticiones paralelas para ir construyendo la interfaz. Primero se carga el scoreboard de Trakt mediante `/api/trakt/scoreboard` para obtener la puntuación, el número de usuarios que están viéndolo actualmente (`watchers`), el número total de reproducciones (`plays`), cuántos usuarios lo tienen en sus bibliotecas (`collectors`) y cuántas listas de Trakt lo incluyen. En paralelo, si el título tiene IMDb ID, se consulta OMDb mediante `/api/omdb` para obtener el rating de IMDb, el número de votos, las puntuaciones de Rotten Tomatoes y Metacritic, y los premios ganados. Todos estos datos se guardan en caché: OMDb en `sessionStorage` con TTL de 24 horas, y el scoreboard de Trakt en memoria con TTL de 5 minutos.

A partir de la API de Plex se detecta si el título está disponible en la biblioteca personal y se guarda durante 7 días en `sessionStorage` y se generan múltiples URLs de reproducción: una para el navegador web (`plexUrl.web`) y otra para la aplicación móvil de Plex (`plexUrl.mobile`).

Los proveedores de *streaming* se obtienen de dos fuentes distintas. Por un lado, TMDb proporciona información básica de la disponibilidad mediante `watch/providers`, indicando si el título está disponible para alquiler, compra o en plataformas de suscripción. Por otro lado, JustWatch (mediante la API no oficial en GraphQL) proporciona enlaces directos al contenido en cada plataforma. Si la película pertenece a una colección (por ejemplo, la saga de *El Señor de los Anillos*), se carga la información completa de la colección mediante `/api/tmdb/collection` y se muestra una

nueva sección con un carrusel y todas las películas de la saga, ordenadas cronológicamente.

Menú de navegación por secciones

El componente `DetailsSectionMenu.jsx` implementa un menú de navegación con pestañas que se adapta dinámicamente en función del número de secciones disponibles que haya y el dispositivo cliente. Para películas, el menú incluye las secciones Media (galería de imágenes y videos), Sentimientos (análisis de pros y contras extraído de comentarios de Trakt mediante IA), Comentarios (reviews de TMDb y comentarios de Trakt), Listas (listas de Trakt que incluyen el título, con pestañas Popular y Trending), Colección (si la película pertenece a una saga), Reparto (reparto completo con fotos y personajes) y Recomendaciones (títulos similares sugeridos por el algoritmo de TMDb y Trakt). Para las series, se añaden las secciones Temporadas y Episodios ya que son exclusivas para este tipo.

Sistema de puntuaciones

El componente `ScoreboardBar.jsx` centraliza las puntuaciones de todas las plataformas en una barra horizontal. Aquí se muestran las puntuaciones de TMDb, Trakt, IMDb (OMDb), Rotten Tomatoes (OMDb) y Metacritic (OMDb) si están disponibles y cada una de ellas se muestra dentro de un badge con el logo de la plataforma, el valor numérico, el número de votos y el enlace externo a la página del título correspondiente de cada plataforma.

La puntuación de IMDb se obtiene de OMDb (`/api/omdb?imdb={imdbId}`) y se muestra junto con el número de votos. Esta petición solo se realiza si el título tiene asociado un IMDb ID en los `external_ids` de TMDb, y el resultado se cachea en `sessionStorage` con una clave específica `omdb:{imdbId}` y un TTL de 24 horas. OMDb también devuelve puntuaciones de Rotten Tomatoes y Metacritic (campos `RottenTomatoes` y `Metascore`).

Además de las puntuaciones, el scoreboard muestra estadísticas de Trakt mediante (`MiniStat`) que incluye iconos descriptivos: el número de usuarios que están viendo el título en este momento (`watchers`, icono Eye), el número total de reproducciones históricas (`plays`, icono Play), cuántos usuarios lo tienen en sus bibliotecas (`collectors`, icono LibraryBig), cuántos comentarios ha recibido (`comments`, icono MessageSquare) y en cuántas listas públicas de Trakt está incluido (`lists`, icono List). Estas estadísticas se cargan mediante `requestIdleCallback` para no bloquear el renderizado inicial del contenido mas relevante.

Botones de acción

La interfaz de detalle incluye varios botones que permiten al usuario gestionar su contenido. A continuación se describe cada uno de ellos:

Botón de Favoritos. Este botón permite marcar películas o series como favoritas en la cuenta de TMDb del usuario. Utiliza la función `markAsFavorite({accountId, sessionId, type, mediaId, favorite})` que realiza una petición POST al endpoint de TMDb `/account/{id}/favorite`. Cuando el usuario hace clic en el botón, el icono cambia inmediatamente de un corazón vacío a un corazón relleno (o viceversa) para proporcionar respuesta visual instantánea. La petición se envía al servidor de TMDb en segundo plano y, si ocurre algún error de red o el servidor rechaza la operación, el icono vuelve a su estado original y se muestra un mensaje de error al usuario. Esta funcionalidad permite al usuario crear una colección personal de contenido que aprecia especialmente.

Botón de Lista de pendientes. Similar al botón de Favoritos, este botón añade o elimina contenido de la lista de pendientes (watchlist) del usuario en TMDb. Funciona mediante la misma API de TMDb pero utilizando el endpoint `/account/{id}/watchlist` con una petición POST. El icono alterna entre un marcador vacío y un marcador relleno según el estado actual. Esta lista sirve para que el usuario organice el contenido que desea ver próximamente pero aún no ha visto, diferenciándolo de sus favoritos que generalmente son contenidos ya vistos y valorados positivamente.

Botón de Valoración personal. Este botón abre una ventana emergente donde el usuario puede asignar una puntuación personal de 1 a 10 (con incrementos de 0.5) al contenido visualizado. La ventana muestra un número grande con la valoración actual sobre una estrella decorativa de fondo, una barra deslizante que permite seleccionar la puntuación deseada, y botones de ajuste fino ($-0{,}5$ y $+0{,}5$) para facilitar la selección precisa en dispositivos táctiles. La valoración se envía a TMDb mediante una petición POST y se almacena en la cuenta del usuario. Si ya existe una valoración previa, se muestra un botón de papelera que permite eliminarla mediante la función `clearTmdbRating()`. Adicionalmente, el componente incluye una opción para sincronizar automáticamente la valoración con Trakt: cuando está activada, al valorar en TMDb también se envía la misma puntuación a Trakt mediante `sendTraktRating(value)`, manteniendo ambas plataformas actualizadas simultáneamente.

Botón de Listas personalizadas. Exclusivo para películas, este botón abre el componente `AddToListModal` que permite gestionar la pertenencia de la película a las listas personalizadas del usuario en TMDb. Al abrirse, el modal carga todas las listas existentes consultando `GET /account/{id}/lists` y verifica en cuáles de ellas está incluida la película mediante peticiones a `GET /list/{listId}/item_status?movie_id={id}`. El resultado se presenta como una lista vertical con casillas de verificación que indican si la película pertenece o no a cada lista. El usuario puede añadir o quitar la película de cualquier lista con un simple clic que ejecuta una petición POST de forma inmediata. También existe la opción de crear una nueva

lista directamente desde este modal rellenando un formulario con nombre y descripción, lo cual ejecuta `POST /list` y, si la creación es exitosa, añade automáticamente la película a la nueva lista. Para usuarios con muchas listas, el modal incorpora un campo de búsqueda que filtra las listas mostradas en tiempo real conforme el usuario escribe.

Botón de Marcar como visto (series). Para series de televisión, este botón gestiona el seguimiento de episodios vistos integrándose con la API de Trakt. Cuando el usuario marca un episodio como visto, se ejecuta `toggleEpisodeWatched()` que envía una petición POST a `/api/trakt/episode/watched` incluyendo el identificador de la serie, número de temporada, número de episodio y la fecha/hora actual. El checkbox del episodio cambia de estado inmediatamente y, si la petición falla, se revierte el cambio mostrando un mensaje de error. Para facilitar el marcado masivo, existe un botón «Marcar temporada completa» que envía todos los episodios no vistos de una temporada en una única petición, evitando múltiples llamadas individuales al servidor.

Botón de Compartir. Este botón permite compartir el enlace de la página de detalles actual mediante las opciones nativas del navegador o del sistema operativo. Utiliza la API Web Share cuando está disponible (principalmente en dispositivos móviles) o copia el enlace al portapapeles como alternativa en navegadores de escritorio que no soporten esta funcionalidad.

Gestión avanzada de episodios para series

El seguimiento episódico es uno de los sistemas más complejos del proyecto. El estado de visionado se representa mediante un mapa anidado `watchedBySeason` con la estructura `{"seasonNumber": {"episodeNumber": true/false}}`, que se construye a partir de la respuesta de Trakt `GET /sync/watched/shows` y se transforma internamente por la API Route `/api/trakt/show/watched`. El componente `SeasonDetailsClient.jsx` renderiza cada temporada como una sección expandible que, al abrirse, carga los detalles de todos los episodios de esa temporada mediante `getSeasonDetails(showId, seasonNumber)` y los muestra en formato grid o lista según la preferencia del usuario (guardada en `localStorage`). Cada episodio se renderiza con su número (`SxE`), nombre, fecha de emisión, puntuación de TMDb, puntuación de Trakt (si está disponible), y un checkbox interactivo que permite marcar/desmarcar el episodio como visto de forma individual.

Cuando el usuario marca un episodio como visto, se dispara la función `toggleEpisodeWatched(seasonNumber, episodeNumber)` que realiza una actualización optimista del checkbox (cambia visualmente a marcado de forma instantánea) y envía una petición POST a `/api/trakt/episode/watched` con el payload `{showId, season, episode, watchedAt: new Date().toISOString()}`. Si la petición falla,

el checkbox se desmarca automáticamente y se muestra un mensaje de error. Si tiene éxito, se recarga el estado completo de la serie mediante `GET /sync/watched/shows/{id}` para sincronizar con cualquier cambio que haya podido ocurrir en Trakt desde otros dispositivos o aplicaciones. Para evitar decenas de peticiones al marcar una temporada completa, existe un botón «Marcar temporada completa» que construye un array con todos los episodios no vistos de esa temporada y envía una única petición batch a Trakt con todos ellos.

El sistema también soporta múltiples rewatches (rewatch runs) mediante un esquema de almacenamiento en `localStorage` con clave `showverse:trakt:show:{showId}:rewatches` que guarda un array de objetos `{id, startedAt, label}`. Cada rewatch run tiene un identificador único generado con timestamp, una fecha de inicio en formato ISO 8601, y un label descriptivo generado automáticamente (por ejemplo, «Rewatch #2 - Marzo 2026»). El usuario puede cambiar entre la vista global (que muestra todos los episodios vistos históricamente sin importar cuándo) y la vista de un rewatch específico (que solo muestra los episodios marcados durante ese run) mediante un dropdown selector. El componente `TraktEpisodesWatchedModal` implementa toda esta lógica en un modal fullscreen responsive que se renderiza como bottom sheet en móvil y como dialogo centrado en desktop.

El componente `EpisodeRatingsGrid.jsx` ofrece una visualización alternativa tipo heat map de todos los episodios de la serie. Cada celda del grid representa un episodio y se colorea con un gradiente según su puntuación de TMDb: verde intenso para episodios excelentes (rating > 8.0), amarillo para episodios medianos (6.0–7.9), rojo para episodios mal valorados (< 6.0), y gris para episodios sin datos de rating. El tamaño de cada celda es deliberadamente pequeño (aproximadamente 12×12 píxeles) para permitir visualizar patrones de calidad a nivel de temporada completa de un solo vistazo: por ejemplo, si una fila de celdas verdes tiene una celda roja, el usuario identifica inmediatamente cuál es el episodio peor valorado de esa temporada sin necesidad de leer ratings numéricos. El grid es completamente interactivo y hacer clic en cualquier celda navega directamente a la página de detalles de ese episodio específico (`/details/tv/{showId}/season/{season}/episode/{number}`).

Para optimizar el rendimiento en series con cientos de episodios (por ejemplo, *The Simpsons* con más de 700 episodios), los ratings se cargan en paralelo mediante `Promise.all(seasons.map(s =>getSeasonDetails(showId, s.season_number)))` en lugar de secuencialmente. Esto reduce la latencia total de aproximadamente N segundos (donde N es el número de temporadas) a aproximadamente 2–3 segundos independientemente del número de temporadas, limitado únicamente por la latencia de red y el rate limiting del servidor.

Modal de enlaces externos y navegación cruzada

El componente `ExternalLinksModal` centraliza todos los enlaces a plataformas externas relacionadas con el título. Al hacer clic en el botón «Enlaces externos», se abre un modal responsive que se renderiza como bottom sheet deslizante en móviles y como diálogo centrado en desktop. El modal muestra hasta ocho tipos de enlaces diferentes, cada uno con su logo característico y su enlace directo. El enlace a la web oficial del título se extrae de los metadatos de TMDb (campo `homepage`) o, si no está disponible, de Trakt mediante `/api/trakt/official-site`. FilmAffinity se construye dinámicamente mediante una búsqueda por título, ya que la plataforma no proporciona una API pública para resolución de IDs.

JustWatch requiere un proceso de resolución más sofisticado. La API `/api/links/justwatch` recibe el título y el año, construye un slug normalizado (título en minúsculas, sin caracteres especiales, espacios convertidos a guiones, año añadido si es necesario para desambiguación), realiza una petición HEAD a la URL candidata para verificar que el recurso existe (código HTTP 200), y cachea el resultado en `localStorage` de forma indefinida. Si la URL directa falla, devuelve una URL de búsqueda como fallback. Para Letterboxd (solo disponible para películas), se utiliza el IMDb ID si está disponible, construyendo la URL como `https://letterboxd.com/imdb/{imdbId}`, o alternativamente se construye un slug basado en el título y se valida su existencia. SeriesGraph (solo para series) recibe directamente el TMDb ID y construye la URL `https://seriesgraph.com/show/{tmdbId}`.

Los enlaces a TMDb, Trakt e IMDb son directos y no requieren resolución: TMDb utiliza el ID numérico del título junto con un slug del título (por ejemplo, `/movie/550-fight-club`), Trakt usa su propio ID obtenido durante la búsqueda inicial del scoreboard (almacenado en el estado como `trakt.traktUrl`), e IMDb simplemente utiliza el IMDb ID en formato `tt1234567`. Todos los enlaces se abren en una nueva pestaña mediante `target="_blank"` con `rel="noopener noreferrer"` para evitar problemas de seguridad. El modal incluye estados de carga individuales por enlace: mientras se resuelve una URL (por ejemplo, JustWatch o Letterboxd), el enlace muestra un spinner de carga y está deshabilitado; si la resolución falla, el enlace desaparece automáticamente del modal.

Gestión avanzada de imágenes y posters

El sistema de gestión de imágenes implementa tres modos de visualización distintos y permite a los administradores seleccionar manualmente qué versión de poster o backdrop se muestra. Para series de televisión, el sistema carga automáticamente las imágenes de TODAS las temporadas mediante `fetchTVImages({showId, apiKey})` que itera por cada temporada, obtiene sus imágenes mediante `GET /tv/{id}/season/{seasonNumber}/images`, y fusiona todos

los resultados en un único array eliminando duplicados por `file_path`. Esto garantiza que el administrador tenga acceso a todas las variantes de poster y backdrop que TMDb tiene disponibles, incluyendo las específicas de cada temporada que a menudo tienen mejor calidad o diseño más actualizado que el poster genérico de la serie.

El sistema distingue tres tipos de imágenes según su función visual. Los posters son imágenes verticales con aspect ratio 2:3 que se muestran en el hero de la página de detalle y se optimizan en dos calidades: `w342` para carga inicial rápida (low quality) y `w780` para carga definitiva de alta calidad. Los backdrops de preview son imágenes horizontales con aspect ratio 16:9 que se utilizan cuando el usuario activa el «modo preview» (toggle en la esquina superior derecha del hero), optimizadas en tamaños `w780` (low) y `w1280` (high). Los backgrounds son backdrops neutrales sin texto que se utilizan como fondo hero en móviles para garantizar legibilidad del contenido superpuesto.

La selección automática de la mejor imagen se realiza mediante funciones de ranking especializadas. Para posters, `pickBestPosterTV(posters, languages)` ordena por resolución (preferencia por imágenes 4K y 2K) y número de votos, priorizando posters en español o inglés si están disponibles. Para backdrops, `pickBestBackdropTVNeutralFirst(backdrops, languages)` prioriza específicamente las imágenes con `iso_639_1 === null` (sin texto superpuesto) para el background de móvil, y luego ordena por resolución y votos para el resto. El sistema implementa carga progresiva: primero carga la versión low quality y la muestra inmediatamente, luego carga la versión high quality en background y, cuando está disponible, realiza un crossfade suave de 500ms mediante transiciones CSS de opacidad.

Los administradores (específicamente el usuario `psantos26` hardcoded en el componente) tienen acceso a una galería administrable con tres pestañas: Posters, Backdrops (para modo preview) y Backgrounds (para fondo móvil). Cada pestaña incluye filtros de resolución (All, 720p, 1080p, 2K, 4K calculados a partir del `width` de cada imagen) y checkboxes de idioma (ES, EN). Las imágenes se muestran en un carrusel horizontal con navigation buttons (flechas izquierda/derecha) implementado con Swiper, y hacer clic en cualquier imagen la establece inmediatamente como activa, guardando la selección en `localStorage` con claves específicas por tipo e ID (por ejemplo, `showverse:movie:550:poster`, `showverse:tv:1399:backdrop`). Las selecciones se sincronizan también con el servidor mediante `POST /api/artwork/override` que guarda las preferencias en el archivo `artwork-overrides.json` para persistencia a largo plazo.

Sección de reparto y ficha de personas

La sección de reparto fusiona datos de múltiples fuentes para proporcionar información completa del cast. Los directores y creadores se extraen del array `credits.crew` filtrando por `job ===`

"Director" o department === "Writing" y se agregan al inicio del array de cast para darles prominencia visual. El array completo se ordena por el campo order que TMDb proporciona para indicar el orden de importancia de cada actor en la producción, de forma que los protagonistas aparecen primero. Cada tarjeta de actor muestra la foto del perfil en tamaño w185, el nombre real del actor, el nombre del personaje que interpreta, y un efecto de hover que aplica escala 105 % y brightness 110 % para indicar interactividad. Hacer clic en cualquier actor navega a la página de ficha de persona (/details/person/{id}) que carga mediante el componente ActorDetails.jsx.

La ficha de persona muestra la biografía completa del actor (campo biography de TMDb que puede contener varios párrafos de texto formateado), su foto de perfil en alta calidad (h632), fecha y lugar de nacimiento, fecha de fallecimiento si aplica, y una filmografía completa dividida en dos pestañas: películas y series de televisión. La filmografía se ordena cronológicamente por release_date o first_air_date en orden descendente (trabajos más recientes primero), y cada entrada muestra el poster del título, el año, el nombre del personaje o rol, y el rating de TMDb. La página también incluye enlaces externos a IMDb (si el actor tiene imdb_id), Instagram, Twitter, Facebook y sitio web oficial, construidos a partir del objeto external_ids de TMDb. Al hacer clic en cualquier título de la filmografía, se navega directamente a la página de detalle de ese título, creando un grafo navegable de contenido relacionado.

Comentarios, análisis de sentimiento y listas de Trakt

La sección de comentarios fusiona reviews de TMDb con comentarios de Trakt en una interfaz unificada con pestañas. Los comentarios de Trakt se organizan en tres categorías: «Top 30 días» que muestra los comentarios con más likes recibidos en los últimos 30 días (filtrado client-side comparando created_at con la fecha actual), «Top histórico» que muestra los comentarios con más likes de todos los tiempos ordenados descendentemente por likes, y «Más recientes» que muestra todos los comentarios ordenados por fecha de creación en orden descendente. Cada comentario incluye el avatar del usuario, su nombre de usuario (clicable para ir al perfil de Trakt), el rating que dio al título si es que votó (escala 1–10 representada con estrellas), una advertencia de spoiler si el comentario está marcado como tal (que oculta el texto hasta que el usuario hace clic explícito en «Mostrar spoiler»), el número de likes que ha recibido, el número de respuestas, la fecha de publicación relativa (por ejemplo, «hace 3 días») y el texto completo del comentario que TMDb devuelve en HTML y se sanitiza mediante dangerouslySetInnerHTML con restricciones de seguridad.

El análisis de sentimiento se construye mediante la función buildSentimentFromComments(items) que procesa todos

los comentarios cargados, extrae frases que contienen palabras clave positivas (como «excelente», «increíble», «obra maestra», «recomendable») o negativas (como «decepcionante», «aburrido», «predecible», «desperdicio»), y genera dos listas: una de pros (aspectos positivos mencionados frecuentemente) y otra de contras (aspectos negativos recurrentes). Cada pro se muestra con un indicador visual positivo (checkmark verde) y cada contra con un indicador visual negativo (X roja). El sistema incluye un contador que indica cuántos comentarios se analizaron para generar el sentimiento, proporcionando contexto sobre la confiabilidad del análisis. Esta funcionalidad es especialmente útil para identificar rápidamente las fortalezas y debilidades de un título sin necesidad de leer decenas de comentarios completos.

La sección de listas muestra las listas públicas de Trakt que incluyen este título, organizadas en dos pestañas: Popular (ordenadas por número de likes) y Trending (ordenadas por actividad reciente). Cada lista se renderiza como una tarjeta que muestra el nombre de la lista, una descripción truncada, el nombre del usuario que la creó, el número de items que contiene, el número de likes que ha recibido y el número de comentarios. Las listas se cargan mediante paginación lazy-loading: inicialmente se muestran 6 listas, y al hacer scroll hasta el final de la sección se cargan automáticamente las siguientes 6 mediante `Intersection Observer`. Hacer clic en cualquier lista navega a la página completa de esa lista en Trakt, abriendo en una nueva pestaña.

Recomendaciones y contenido similar

La sección de recomendaciones utiliza el algoritmo de TMDb para sugerir títulos similares basándose en género, palabras clave, directores, actores y otros metadatos. Los datos se obtienen server-side mediante `GET /{type}/{id}/recommendations` y se pasan como prop al componente cliente. El renderizado se realiza mediante un carrusel Swiper completamente responsive que ajusta automáticamente el número de slides visibles según el viewport: 3 slides en móviles, 4 en tablets pequeñas, 5 en tablets grandes, 6 en laptops y hasta 7 en pantallas 4K. Cada tarjeta de recomendación muestra el poster del título en tamaño `w342`, el título, el año de estreno y el rating de TMDb.

El sistema implementa carga bajo demanda de ratings de IMDb para las recomendaciones. Inicialmente, las tarjetas solo muestran el rating de TMDb (que ya viene en los datos de la prop). Cuando el usuario hace hover sobre una tarjeta, se dispara la función `prefetchRecImdb(rec)` con un delay de 180ms para evitar peticiones excesivas si el usuario está moviendo rápidamente el cursor. Esta función comprueba primero si el título tiene IMDb ID en `external_ids`, luego busca en el cache de `localStorage` con clave `omdb:{imdbId}`, y si no encuentra datos cacheados, realiza una petición a `/api/omdb?imdb={imdbId}` y cachea el resultado durante 24 horas. Una vez cargado, el rating de IMDb se muestra como un badge adicional en la tarjeta con animación

de fade-in. Hacer clic en cualquier tarjeta navega a la página de detalle de ese título mediante `router.push(`/details/{type}/{id}`)` preservando el estado de navegación.

4.4.3 3. Seguimiento episódico (series)

Complejidad: muy alta. Uno de los sistemas más complejos del proyecto.

Representación de estado: el visionado por episodio se indexa en `watchedBySeason`:

```
{
  "1": { "1": true, "2": true, "3": false },
  "2": { "1": false, "2": true },
  ...
}
```

Obtenido de Trakt mediante `GET /sync/watched/shows/{id}`, transformado internamente por la API Route `/api/trakt/show/watched`.

4.4.4 4. Favoritos (`/favorites`)

La página de favoritos es uno de los módulos más ricos en términos de interacción y personaje dentro de *The Show Verse*. La colección personal del usuario se carga desde TMDb mediante los *endpoints* `GET /account/{id}/favorite/movies` y `/account/{id}/favorite/tv`, pero el desafío real no está en obtener los datos sino en orquestarlos eficientemente cuando un usuario puede acumular cientos de títulos distribuidos en múltiples páginas de 20 ítems. El componente `FavoritesClient.jsx` resuelve esto con un patrón de descarga progresiva: primero solicita la página 1 para conocer el valor de `total_pages`, y a continuación lanza todas las páginas restantes en paralelo mediante `Promise.allSettled()`, limitando la concurrencia máxima a 5 peticiones simultáneas para respetar los *rate limits* de TMDb. El resultado es que una colección de 100 títulos distribuida en 5 páginas se descarga en el tiempo que tardaría una sola petición paralela, en lugar de acumularse secuencialmente.

Una vez cargados, los ítems se enriquecen de forma progresiva mediante una capa de puntuaciones externas gestionada con un sistema de caché multinivel. Las puntuaciones de IMDb y OMDb se cachean en `sessionStorage` con una clave por `imdbId` y un tiempo de vida de 24 horas, de modo que si el usuario navega a favoritos, luego a watchlist, y vuelve a favoritos, los *ratings* ya cargados no requieren ninguna petición adicional. Para prevenir que múltiples tarjetas soliciten simultáneamente el mismo `imdbId` (por ejemplo, si dos películas de la misma saga comparten puntuaciones), el componente implementa un mapa de peticiones en vuelo

(`traktScoreInFlight`) que garantiza que aunque cinco tarjetas soliciten el mismo dato al mismo tiempo, solo se realice una petición real y todas esperen su resultado compartido. Las puntuaciones de Trakt se cachean adicionalmente en `localStorage` con un TTL de 24 horas y marcas de tiempo en formato epoch, lo que las hace persistentes entre sesiones sin necesitar una nueva petición de red al día siguiente. Para no penalizar el tiempo de carga inicial, todas estas puntuaciones externas se cargan de forma perezosa únicamente cuando el usuario hace hover sobre una tarjeta, evitando decenas de peticiones en el montaje del componente.

La interfaz ofrece tres modos de visualización completamente distintos cuya selección persiste entre sesiones en `localStorage`. El modo *grid* organiza las tarjetas en 6 columnas en escritorio, 4 en tablet y 2 en móvil, mostrando únicamente el poster, el título y el rating. El modo *list* muestra filas horizontales con más información contextual: sinopsis truncada, año de estreno y géneros. El modo *compact* optimiza la densidad de información con una línea por ítem, ideal para colecciones de centenares de títulos. Sobre esta base se añaden filtros cruzados por género (con detección automática de géneros presentes en la colección), tipo de contenido (películas, series), rango de valoración y año, más una búsqueda en tiempo real con normalización de acentos mediante la función `normText()` que garantiza que búsquedas como «peliculas» encuentren también «películas». Las tarjetas incorporan un sistema de selección inteligente de imagen que prefiere backdrops de resolución 1920×1080 para la vista *list*, y posters en inglés con mayor número de votos para la vista *grid*, con múltiples niveles de *fallback* hasta el `poster_path` original de TMDb.

4.4.5 5. Pendientes/Watchlist (`/watchlist`)

El módulo de watchlist comparte la misma arquitectura técnica que Favoritos: idéntico patrón de descarga paralela por páginas, el mismo sistema de caché multinivel para puntuaciones externas y las mismas tres vistas con persistencia en `localStorage`. La diferencia fundamental es semántica: mientras que favoritos agrupa el contenido que el usuario ya ha consumido y valora especialmente, la watchlist recoge el contenido que quiere ver en el futuro. Esta distinción semántica se refleja en la interfaz con iconografía diferenciada (marcadores Bookmark en lugar de corazones Heart) y en el comportamiento al eliminar un ítem, que envía una petición `POST /api/tmdb/watchlist` con el parámetro `watchlist: false` en lugar del equivalente de favoritos.

Una característica importante de la watchlist es el patrón de actualización optimista aplicado a la operación de eliminación: cuando el usuario hace clic en el botón de eliminar, el ítem desaparece visualmente de manera instantánea del listado mediante una animación de salida de Framer Motion, y la petición de eliminación real a TMDb se envía en paralelo sin bloquear la interfaz. Si la petición falla por cualquier motivo, el ítem reaparece automáticamente con una notificación de error. Tanto

Favoritos como Watchlist comparten el mismo módulo de caché de puntuaciones, lo que significa que si el usuario ha cargado los ratings de OMDb en la página de favoritos, al navegar a watchlist esos mismos ratings están disponibles instantáneamente desde `sessionStorage` sin ninguna petición adicional.

4.4.6 6. Historial de visionado (`/history`)

El historial de visionado se alimenta directamente de Trakt mediante `GET /sync/history` porque es la única fuente que registra el *timestamp* exacto de cada visionado en formato ISO 8601, incluyendo la hora concreta de la noche, e incluso el episodio específico en el caso de series. La API Route interna `/api/trakt/history` resuelve el principal problema estructural de los datos de Trakt: sus respuestas contienen solo el identificador interno de Trakt, sin imágenes ni metadatos que reconozca el *frontend*. Para cada ítem del historial, la API Route traduce el identificador de Trakt al correspondiente TMDb ID usando el *endpoint* `/search/tmdb/{traktId}?type=movie|show` de Trakt, y lanza en paralelo una petición a TMDb para obtener el `poster_path`, el `backdrop_path` y la sinopsis del título. El resultado llega ya enriquecido al cliente, que no necesita hacer ninguna petición adicional para mostrar la colección completa con sus imágenes.

En el cliente, el componente `HistoryClient.jsx` implementa un sistema de agrupación temporal con cuatro granularidades: por día, por semana, por mes y por año, con cabeceras separadoras que contextualizan temporalmente cada grupo. Para series, aplica la función `collapseConsecutive()` que detecta episodios consecutivos de la misma serie vistos en la misma sesión y los colapsa en una entrada única con notación de rango, por ejemplo «T1E1– T1E5», reduciendo drásticamente la densidad visual del historial cuando el usuario ha visto varios episodios seguidos. Cada entrada del historial muestra el badge de episodio en formato «T01 · E01», el timestamp relativo (por ejemplo, «hace 3 días») y un enlace directo al episodio exacto en la página de detalle de la serie. Para manejar historiales muy extensos sin degradar el rendimiento, el componente utiliza un `IntersectionObserver` con un margen de 200 px para detectar cuando el usuario se aproxima al final de la lista y cargar automáticamente la siguiente página mediante el hook `useInView()`.

4.4.7 7. En Progreso (`/in-progress`)

La página de series en progreso implementa una lógica de cálculo específica para determinar qué series considera «activas»: una serie se clasifica como en progreso si y solo si tiene al menos un episodio marcado como visto pero no todos los episodios disponibles han sido completados. Esta distinción se realiza cruzando la respuesta de `GET /sync/watched/shows` de Trakt, que

contiene el número de episodios vistos por temporada, con los datos de TMDb que informan del número total de episodios emitidos hasta la fecha. El porcentaje de completitud se calcula como `pct = (episodiosVistos / episodiosTotalesEmitidos) * 100`, ignorando episodios futuros no emitidos. El componente también gestiona una pestaña paralela de series completadas, que agrupa los títulos en los que todos los episodios disponibles ya han sido marcados como vistos.

La visualización principal de cada serie en progreso integra tres elementos clave. Un indicador de progreso circular implementado en SVG puro con animación `stroke-dasharray` de Framer Motion codifica el porcentaje de completitud mediante un sistema de colores que escala del rojo rosado (menos del 10 %) al verde esmeralda (más del 90 %), pasando por naranja, ámbar, azul cielo y violeta para los rangos intermedios. Esto permite al usuario valorar de un vistazo qué series están apenas comenzadas y cuáles están a punto de terminar. Debajo del indicador, el componente muestra el código del próximo episodio no visto (por ejemplo, «T02E05») con un botón «Continuar viendo» que construye dinámicamente la URL de la página de detalles del episodio correspondiente. El modo compacto sustituye el indicador circular por una barra de progreso horizontal con el texto fraccionario «30 / 62 episodios», priorizando la densidad de información sobre el impacto visual.

4.4.8 8. Estadísticas del usuario (`/stats`)

El módulo de estadísticas es el panel de análisis más complejo de la aplicación. En lugar de consultar múltiples *endpoints* independientes, todas las estadísticas se obtienen mediante una única petición a la API Route interna `/api/trakt/user-stats`, que agrega internamente la información de `/users/me/stats` (resumen global del usuario), el historial completo de `/sync/history` (para calcular distribuciones temporales precisas) y las listas de `watchedMovies` y `watchedShows` (para identificar los títulos más vistos). La respuesta consolidada incluye totales de películas, series, episodios, temporadas, tiempo en red y distribución de géneros, así como listas de los actores y directores más frecuentes en el historial del usuario.

En el cliente, el componente `StatsClient.jsx` transforma estos datos brutos en un conjunto de visualizaciones interactivas mediante la librería Recharts, todas ellas computadas como valores derivados mediante `useMemo` para evitar recalcular al re-renderizar. La sección principal presenta tarjetas KPI con el total de horas de visionado, películas vistas, series seguidas y episodios completados. El gráfico de actividad mensual es un `AreaChart` que muestra los últimos 12 meses, pre-inicializando a cero los meses sin actividad para que la línea sea continua. La distribución por día de la semana y por hora del día se representa mediante `BarChart` que revelan patrones de consumo como «el usuario ve más contenido los viernes por la noche». La distribución de géneros se visualiza como un `RadarChart` que mapea los seis géneros más consumidos en un polígono regular, permitiendo identificar de forma intuitiva si el usuario prefiere dramas, comedias o thrillers.

El histograma de ratings (distribución de notas 1–10 dadas por el usuario) adopta la forma de un `BarChart` horizontal que puede revelar tendencias como «el usuario nunca pone notas por debajo de 5». La página se organiza en tres vistas seleccionables: resumen general, patrones temporales y desglose anual, con la selección activa gestionada por el estado `viewMode`.

4.4.9 9. Calendario de estrenos (`/calendar`)

El calendario de estrenos combina dos fuentes de datos según el estado de autenticación del usuario. Para todos los usuarios, se cargan los estrenos generales a partir de los *endpoints* de TMDb `/movie/upcoming` y `/tv/on_the_air`. Para los usuarios autenticados con Trakt, se añade la fuente personalizada `/calendars/my/shows` que devuelve los próximos episodios específicamente de las series que el usuario sigue, lo que convierte el calendario en una herramienta de seguimiento activo en lugar de un simple listado de estrenos globales.

La interfaz implementa tres modos de visualización seleccionables mediante botones y con persistencia en `localStorage`. La vista de día muestra qué estrena en la fecha seleccionada mediante el componente `react-day-picker` personalizado con estilos Tailwind; las fechas con contenido se marcan con un indicador visual amarillo en el calendario, y al seleccionar una fecha se actualiza el panel derecho con los títulos de ese día. La vista de semana amplía la ventana a siete días con filas separadas por día, lo que permite planificar qué ver durante la semana. La vista de mes renderiza un calendario de cuadrícula clásico similar a Google Calendar, donde cada celda muestra miniaturas de los posters del contenido que estrena ese día. La navegación entre períodos (día anterior/siguiente, semana anterior/siguiente, mes anterior/siguiente) se implementa con funciones utilitarias de `date-fns` con localización española, y cada cambio de período desencadena una nueva petición a la API Route interna parametrizada por rango de fechas en formato `YYYY-MM-DD`.

4.4.10 10. Biblioteca Personal (`/biblioteca`)

La biblioteca personal es el módulo que más directamente expone la integración con Plex Media Server. A diferencia de otras páginas que trabajan con datos de TMDb o Trakt, `BibliotecaClient.jsx` consulta directamente la API interna `/api/plex` para obtener el inventario completo de la biblioteca local del usuario, incluyendo metadatos técnicos que solo Plex conoce: la resolución de cada archivo (4K/1080p/720p/576p/SD), el códec de vídeo (H.264, HEVC/H.265, AV1), el contenedor (`mkv`, `mp4`, `avi`), el tamaño en disco en GB, y el número de pistas de audio y subtítulos disponibles. Esta información, invisible en plataformas de *streaming*, resulta especialmente útil para cinéfilos que gestionan su propia colección y necesitan saber qué versión de un título tienen almacenada.

El rendimiento es el mayor desafío de este módulo porque las bibliotecas Plex pueden contener miles de títulos. Para no bloquear el hilo principal renderizando 3000 tarjetas al mismo tiempo, el componente implementa renderizado incremental: renderiza los primeros 180 ítems de forma inmediata (`INITIAL_RENDER_COUNT = 180`) y añade bloques adicionales de 120 ítems (`RENDER_CHUNK_SIZE = 120`) cada vez que el usuario hace *scroll* hasta el final de la lista, con prefetch de imágenes 80 ítems por delante de lo actualmente visible (`ARTWORK_PREFETCH_AHEAD = 80`). Los datos de TMDb asociados a cada título (poster y backdrop en alta calidad) se cachean en un Map a nivel de módulo con deduplicación de peticiones en vuelo, y las preferencias de imagen del administrador se leen de `localStorage` con clave `showverse:movie:{id}:poster` antes de recurrir a la selección automática. La interfaz incluye un sistema de apertura directa en Plex mediante la función `openPlexLink()` que detecta la plataforma del usuario y genera la URL apropiada: `intent://` para Chrome en Android, el esquema `plex://` para la app nativa en iOS, o la URL web para escritorio.

4.4.11 11. Descubrir contenido (`/discover`)

El motor de descubrimiento es el módulo con mayor número de parámetros de filtrado de toda la aplicación. A diferencia de la búsqueda libre por texto que utiliza el *endpoint* `/search/multi`, Discover emplea el *endpoint* estructurado `/discover/movie` y `/discover/tv` de TMDb, que acepta parámetros de filtrado ortogonales y combinables. El componente `DiscoverClient.jsx` gestiona el estado completo de los filtros activos: género (multiselección mediante chips), plataforma de *streaming* disponible (con logos de JustWatch), certificación de edad (`G`, `PG`, `PG-13`, `R`, etc.), rango de valoración media y mínimo de votos (mediante sliders de doble recorrido con debounce de 500 ms para evitar peticiones en cada movimiento del deslizador), rango de fechas de estreno y palabras clave free-text con autocompletado contra el *endpoint* `/search/keyword` de TMDb. Todos estos filtros se combinan en una única petición estructurada a TMDb cuyos parámetros se calculan dinámicamente mediante la función `buildDiscoverParams()` a partir del estado activo.

El componente incluye además un modo «no visto» que cruza los resultados del *endpoint* de descubrimiento con la colección de películas y series que el usuario tiene valoradas en Trakt, excluyendo de los resultados los títulos que ya han sido puntuados bajo la premisa de que una puntuación implica haberlo visto. Cada cambio de filtro o de tipo de contenido (película/serie) cancela inmediatamente la petición anterior mediante `AbortController` antes de lanzar la nueva, evitando condiciones de carrera en las que una petición lenta de una búsqueda anterior sobreescriba los resultados de la búsqueda actual. La paginación funciona mediante *infinite scroll*: un `IntersectionObserver` detecta cuando el usuario llega al final del listado y carga

automáticamente la siguiente página añadiéndola al array existente en lugar de sustituirlo, creando la ilusión de un listado continuo e infinito. Los resultados se presentan en las mismas tres vistas (grid, list, compact) que el resto de módulos de colección, con animaciones de entrada escalonadas de Framer Motion para cada lote de nuevas tarjetas añadidas al scroll.

4.4.12 12. Gestión de listas (`/lists`)

El módulo de listas unifica cuatro tipologías de contenido curatorado en una sola interfaz. Las listas propias del usuario en TMDb se obtienen mediante `GET /account/{id}/lists` y permiten operaciones completas de CRUD: el usuario puede crear nuevas listas rellenando un formulario modal con nombre y descripción (petición `POST /list`), añadir o eliminar títulos desde la propia página de detalle, y editar los metadatos. Las colecciones de saga de TMDb (franquicias como el MCU, la saga de Bourne o el universo de Star Wars) se tratan como un tipo especial de lista que el componente `CollectionDetailsClient.jsx` renderiza con un hero propio y un grid ordenado cronológicamente. Las listas públicas de Trakt se gestionan mediante el componente `TraktListDetailsClient.jsx`, que implementa paginación con carga bajo demanda en páginas de 48 ítems mediante un botón «Cargar más» que consulta el *endpoint* paginado `/api/trakt/lists/{username}/{listId}?page={n}&limit=48`.

La página principal del módulo (`/lists`) precargar automáticamente los detalles de las primeras 3 listas del usuario (`PRELOAD_FIRST_N_LISTS = 3`) para que al hacer clic sobre ellas la navegación sea instantánea. Las listas se muestran como tarjetas con un collage compuesto de hasta cuatro backdrops de los primeros títulos de la lista organizado en variantes de cuadrícula (1×1 para listas con un título, 2×1 para dos, 2×2 para tres o más), generado por la función `ListCoverBackdropCollage()`. El diseño visual de cada lista incluye metadatos de interés como el número de ítems, el nombre del creador y la descripción truncada, y permite ordenar las listas del usuario por fecha de creación, número de ítems o valoración media del contenido. El componente `UnifiedListDetailsLayout.jsx` actúa como envoltorio de maquetación reutilizable para todas las páginas de detalle de lista, proporcionando el hero con título y metadatos, el sistema de pestañas con estado activo, y los controles superiores de filtrado y ordenación de forma consistente independientemente de si la lista es de TMDb, una colección o una lista de Trakt.

4.4.13 13. Películas y Series (`/movies`, `/series`)

Las páginas de películas y series son extensiones temáticas del dashboard principal, especializadas en un único tipo de contenido. `MoviesPageClient.jsx` y `SeriesPageClient.jsx` comparten la misma arquitectura técnica, incluyendo sistemas de caché a nivel de módulo para extras (`movieExtrasCache`), backdrops (`movieBackdropCache`), imágenes

(`movieImagesCache`) y trailers (`movieTrailerCache` con mapa de peticiones en vuelo para deduplicación). La función `pickBestBackdropByLangResVotes()` prioriza backdrops de resolución 1920×1080 sobre 2560×1440 y 3840×2160, priorizando la velocidad de carga sobre la calidad máxima de imagen en estos contextos de navegación. La selección de poster aplica `pickBestPosterByLangThenResolution()` que calcula el área de cada imagen disponible (ancho × alto) y selecciona el de mayor resolución entre los disponibles en inglés o español, con *fallback* al `poster_path` original de TMDb.

La sección de trailers permite al usuario reproducir el tráiler oficial de cualquier título directamente en la página sin navegar al detalle. Cuando el usuario hace hover sobre una tarjeta, la función `prefetchTrailer(id)` comprueba el caché de trailers en memoria y, si no hay datos cacheados, solicita los vídeos del título a TMDb mediante `GET /{type}/{id}/videos` y almacena el resultado. Al hacer clic en el botón de reproducción, el componente abre el `VideoModal` con el `videoKey` de YouTube del tráiler, evitando que el usuario tenga que navegar a la página de detalle únicamente para ver el tráiler. Ambos módulos incluyen integración directa con los botones de favorito y watchlist de TMDb desde las tarjetas del grid, usando el mismo patrón de actualización optimista descrito en el módulo de detalles, sin necesidad de navegar a la página de detalle para gestionar las listas personales.

4.4.14 14. Ficha de personas (`/details/person/[id]`)

La ficha de persona implementada en `ActorDetails.jsx` carga en paralelo mediante `Promise.allSettled()` cinco recursos independientes: los identificadores externos (`/person/{id}/external_ids`), los créditos combinados de películas y series (`/person/{id}/combined_credits?language=es-ES`), la galería de fotos de perfil (`/person/{id}/images`), las imágenes promocionales etiquetadas (`/person/{id}/tagged_images`) y las traducciones disponibles (`/person/{id}/translations`), escogiendo la versión en español de la biografía si está disponible. El uso de `Promise.allSettled()` en lugar de `Promise.all()` garantiza que un fallo en uno de los recursos (por ejemplo, ausencia de imágenes etiquetadas) no cancele la carga de los demás.

La filmografía completa se computa como un valor derivado mediante `useMemo` a partir del array `combined_credits`, normalizando el formato heterogéneo de los campos según si el ítem es una película (`release_date`) o una serie (`first_air_date`), y si el crédito es como actor (`character`) o como miembro del equipo técnico (`job`, `department`). Los filtros disponibles son: tipo de medio (todas, películas, series), tipo de crédito (actuación, equipo técnico), departamento (para filtrar directores, guionistas, productores dentro del equipo técnico), rango de

años desde/hasta, y búsqueda libre por título o nombre de personaje. Todos los filtros se aplican de forma combinada en un único paso de procesamiento sobre el array completo, sin peticiones adicionales al servidor. La ordenación por popularidad, fecha, valoración o alfabético se aplica como una etapa posterior al filtrado. El sistema calcula estadísticas sobre la filmografía filtrada (total de créditos, películas, series, actuaciones, roles técnicos) que se muestran como tarjetas informativas en la cabecera, actualizándose en tiempo real al cambiar los filtros. La función auxiliar `calcAge()` calcula la edad actual o la edad en el momento del fallecimiento a partir de las fechas de nacimiento y muerte, y `formatHumanDate()` formatea las fechas en español con localización `es-ES`.

4.4.15 15. Barra de navegación (Navbar)

La barra de navegación implementada en `Navbar.jsx` es el único componente que está presente en absolutamente todas las páginas de la aplicación, ya que se instancia directamente en el `layout.jsx` raíz. En escritorio adopta una disposición horizontal en tres zonas: el logotipo y nombre de la aplicación a la izquierda, una barra de búsqueda centralizada, y a la derecha un conjunto de iconos de navegación con códigos de color por categoría temática (estadísticas en índigo, listas en morado, calendario en ámbar, series en progreso en verde, historial en rojo, favoritos en rojo, watchlist en azul). En móvil, la barra superior simplifica el contenido a logo, buscador y avatar, mientras que la navegación principal se traslada a una barra fija de iconos en la parte inferior de la pantalla que siempre permanece accesible con el pulgar, complementada por un menú deslizable lateral (*drawer*) que contiene los enlaces secundarios.

La barra de búsqueda es un componente independiente `SearchBar` reutilizado tanto en la versión de escritorio como en la modal de búsqueda de móvil. Implementa un debounce de 300 ms antes de lanzar cada consulta a `/search/multi?query=...&language=es-ES`, y cancela las peticiones anteriores con `AbortController` para evitar que resultados de búsquedas superadas sobrescriban los de la búsqueda actual. El *dropdown* de resultados muestra hasta diez sugerencias con el tipo de contenido codificado por color (badge azul para películas, badge verde para series, badge naranja para personas), la foto de perfil o poster en miniatura, el año de estreno y el rating de TMDb. Una variante del componente Navbar incluye el botón `TraktHistoryNavButton` que muestra en tiempo real cuántos ítems hay en la lista de pendientes, obtenido del contexto de autenticación `AuthContext` sin petición adicional. El efecto de transición de transparente a *glassmorphism* al hacer *scroll* se implementa con un listener `scroll` que aplica o elimina la clase `bg-black/70 backdrop-blur-12px` al superar un umbral de 100 px.

4.4.16 Patrones de optimización transversales

Los módulos funcionales de *The Show Verse* comparten un conjunto de patrones de optimización que se aplican de forma sistemática y que explican los tiempos de respuesta percibidos como inmediatos incluso siendo una aplicación que integra datos de seis APIs externas distintas.

El patrón de actualización optimista se aplica en todas las acciones de escritura del usuario: al hacer clic en el corazón de favoritos, en el marcador de watchlist o en la estrella de valoración, el estado visual de la interfaz se actualiza instantáneamente sin esperar la confirmación del servidor. La petición real se envía en paralelo en segundo plano, y solo si esta falla por cualquier motivo (error de red, sesión expirada, error del servidor) el estado visual se revierte automáticamente a su valor anterior mostrando un mensaje de error. Este patrón elimina la latencia perceptible de 300 a 600 ms que tendría una actualización sincrónica, creando la sensación de respuesta inmediata que los usuarios esperan de aplicaciones nativas.

El sistema de caché multinivel para puntuaciones externas cubre todos los casos de reutilización. Las puntuaciones de OMDb e IMDb se cachean en `sessionStorage` con TTL de 24 horas y una clave por `imdbId`, de modo que persisten durante toda la sesión del navegador pero se renuevan en la siguiente. Las puntuaciones de Trakt se cachean en `localStorage` con TTL de 24 a 48 horas y marcas de tiempo epoch, con lo que sobreviven al cierre y reapertura del navegador. A nivel de peticiones en vuelo, cada módulo mantiene un `Map` de promesas pendientes (`inFlight`) por identificador de recurso que garantiza que si diez tarjetas solicitan el mismo `imdbId` simultáneamente, solo se ejecute una petición real y todas compartan su resultado cuando llegue. Las imágenes de poster y backdrop se precargan mediante promesas con un adelanto configurable respecto al viewport (`ARTWORK_PREFETCH_AHEAD`), de forma que cuando el usuario hace *scroll* las imágenes ya están en la caché del navegador.

La función utilitaria `runPool(items, limit, worker)` implementa el patrón de pool de concurrencia que aparece en casi todos los módulos: ejecuta la función `worker` para cada elemento del array `items` manteniendo en todo momento exactamente `limit` operaciones concurrentes (típicamente 4 o 5), lo que respeta los *rate limits* de las APIs externas sin renunciar al paralelismo. Este patrón reduce el tiempo de carga de colecciones paginadas de N peticiones secuenciales a $\lceil N/\texttt{limit} \rceil$ rondas paralelas, un factor de mejora de 4–5× respecto a la alternativa secuencial. Cuando el usuario navega a una nueva página mientras hay peticiones en vuelo, todos los módulos cancelas sus `fetch` activos mediante `AbortController` en la función de limpieza del `useEffect`, previniendo tanto condiciones de carrera como actualizaciones de estado en componentes ya desmontados que generarían errores en React. Finalmente, los datos considerados semi-estables (catálogo de tendencias, listas top rated) se sirven desde el servidor con ISR y `revalidate` entre 600 y 1800 segundos, haciendo que la mayoría de visitas sirvan páginas

preconstruidas desde la CDN de Vercel sin tocar ninguna API externa.

4.5 Diseño de la interfaz

4.5.1 Sistema de diseño global

El sistema de diseño de *The Show Verse* parte de un principio rector: el contenido visual proveniente de TMDb (los *backdrops* y los *posters* de cada título) es el protagonista de la interfaz. La paleta de colores, los efectos de superficie y las estrategias de iluminación se han diseñado para potenciar, no competir con, esas imágenes.

Paleta y color de base. El color de fondo global es el negro puro (`#000000`), definido en el `layout.jsx` raíz. Sobre este fondo se despliegan los *backdrops* a pantalla completa con un gradiente perimetral que los integra suavemente en el negro sin corte brusco. Este enfoque traslada la estética de las propias plataformas de *streaming* (modo teatro) a la interfaz de seguimiento.

Superficies de cristal (*glassmorphism*). Las tarjetas, paneles laterales, la Navbar con *scroll* y los *dropdowns* de búsqueda utilizan el patrón definido en la clase utilitaria `.glass` del fichero `globals.css`:

```
.glass {
  background: rgba(255, 255, 255, 0.05);
  backdrop-filter: blur(10px);
  -webkit-backdrop-filter: blur(10px);
  border: 1px solid rgba(255, 255, 255, 0.1);
}
```

La opacidad del fondo (`0.05`) es deliberadamente baja para mantener la legibilidad del texto proyectando el color del *backdrop* subyacente en la superficie del elemento. El desenfoque de contexto (`backdrop-filter: blur`) es processing GPU-acelerado y no genera coste de repintado del DOM.

Tipografía. El proyecto usa la familia Geist Sans y Geist Mono, tipografías de Vercel distribuidas como variable CSS mediante el módulo `next/font`. Se cargan como fuentes de sistema sin peticiones externas, evitando el FOUT (*Flash of Unstyled Text*).

Accesibilidad y movimiento reducido. El fichero `globals.css` incluye una media query `prefers-reduced-motion` que desactiva todas las transiciones y animaciones de CSS para usuarios que han solicitado minimizar el movimiento en su sistema operativo, sin modificar ningún componente individual.

4.5.2 Componentes de Next.js utilizados

`<Image>` de Next.js. Todos los pósters, *backdrops* y logos de la aplicación se renderizan mediante el componente `<Image>` de Next.js en lugar del elemento `<img>` nativo. Este componente aplica automáticamente:

- Conversión del formato original JPEG a WebP para los navegadores que lo soportan, reduciendo el peso de las imágenes entre un 25 y un 35 % sin pérdida visual apreciable.
- *Lazy loading* nativo: las imágenes fuera del *viewport* no se solicitan hasta que el usuario se aproxima a ellas.
- Reserva de espacio (*placeholder*) mediante `width` y `height` explícitos, evitando el salto de layout (CLS) que se produciría si la imagen tardase en cargar.
- Optimización automática a través del CDN de Vercel, que sirve cada imagen en el tamaño exacto requerido por el dispositivo del usuario.

`<Link>` de Next.js. La navegación entre páginas utiliza el componente `<Link>` del App Router de Next.js, que implementa *prefetching* automático de las rutas visibles en el *viewport*. En producción (Vercel), cuando el usuario se aproxima a un enlace, Next.js precarga en segundo plano los datos del Server Component de la ruta destino, logrando transiciones de página percibidas como instantáneas.

`usePathname` y estado activo en la Navbar. El componente `Navbar.jsx` utiliza el *hook* `usePathname()` de `next/navigation` para detectar la ruta activa y aplicar el estilo visual correspondiente a cada enlace de navegación sin necesidad de estado global. La función `isActive(href)` comprueba si la ruta actual comienza con el *href* del enlace, marcando toda la sección como activa aunque el usuario esté en una subruta.

4.5.3 Componentes propios: Navbar

El componente `Navbar.jsx` (30 KB) implementa una navegación adaptativa con dos modos de presentación.

Modo escritorio (lg+). Una barra horizontal fija en la parte superior con tres zonas:

- **Izquierda**: logotipo de la aplicación y navegación principal (Inicio, Películas, Series, Descubrir, Biblioteca).
- **Centro** (posición absoluta): campo de búsqueda en tiempo real con *dropdown* de resultados.

- **Derecha**: botones de acceso rápido a Estadísticas, Listas, Calendario, En Progreso, Historial, Favoritas y Pendientes; avatar del usuario autenticado.

Cada botón de icono utiliza un sistema de estilos por «tono de color» (`iconLinkClass(href, tone)`): al hacer *hover*, el botón adquiere el color semántico de su función (rojo para Favoritas, azul para Pendientes, ámbar para Calendario, verde para En Progreso, etc.) junto con un halo de sombra coloreado:

```
hover:shadow-[0_0_18px_rgba(239,68,68,0.16)]
```

Cuando el estado está activo (ruta actual), el color se intensifica con un fondo semitransparente y una sombra más pronunciada.

Modo móvil. La barra superior se simplifica al logotipo centrado, un botón de menú (hamburgesa) y un botón de búsqueda. En la parte inferior de la pantalla aparece una barra de navegación fija (*tab bar*) con las cuatro secciones de mayor uso. Esta arquitectura de doble barra es el patrón estándar de las aplicaciones de contenido multimedia en iOS y Android.

El **drawer lateral** del menú móvil se implementa con Framer Motion:

```
<motion.aside
  initial={{ x: -320 }}
  animate={{ x: 0 }}
  exit={{ x: -320 }}
  transition={{ type: "tween", duration: 0.22 }}
>
```

El fondo oscuro detrás del drawer (`bg-black/70 backdrop-blur-sm`) también se anima con `AnimatePresence` para que tanto la entrada como la salida del menú sean fluidas.

La **búsqueda en móvil** abre un *overlay* a pantalla completa (`bg-black/95 backdrop-blur-2xl`) que entra con una animación de escala y opacidad (`scale: 0.95→1, opacity: 0→1`), dando la sensación de que emerge desde el botón de búsqueda.

4.5.4 Componente propio: LiquidButton

`LiquidButton.jsx` (18 KB) es el componente de botón de acción rápida utilizado en la página de detalle para las operaciones de favoritos, pendientes, historial y valoración personal. Es un Client Component completamente autónomo que implementa un sistema de efectos visuales interactivos mediante la API `Canvas 2D` del navegador.

La arquitectura del componente se basa en tres capas superpuestas dentro de un `<button>`:

1) **Canvas de efectos**: un elemento `<canvas>` que ocupa toda la superficie del botón y gestiona, mediante `requestAnimationFrame`, las partículas flotantes, las ondulaciones de *ripple* y los efectos de explosión al hacer clic.

2) **Superposición de brillo**: un `<div>` con un gradiente lineal animado mediante CSS (`@keyframes liquidShine`) que recorre la superficie del botón de forma continua cuando está activo o en *hover*.

3) **Borde pulsante**: un segundo `<div>` circular con borde coloreado animado mediante `@keyframes liquidPulse` (escala $1 \rightarrow 1.15 \rightarrow 1$, con periodo de 2 s).

El sistema de efectos de *canvas* implementa:

- **Partículas flotantes**: 12 partículas con movimiento senoidal suave que se generan al entrar en *hover*. Cada partícula tiene posición, velocidad, tamaño, opacidad y fase trigonométrica independientes.

- **Ripple (ondulación)**: al hacer *hover* o clic, se crean ondas circulares que se expanden desde el punto de contacto y se desvanecen al alcanzar su radio máximo.

- **Trail effect**: al mover el ratón sobre el botón, se generan puntos de rastro que se desvanecen en 0,4 s, creando un efecto de «estela líquida».

- **Explosión**: al hacer clic, se generan 12 partículas que se dispersan en ángulos uniformes desde el punto de impacto con gravedad simulada (`vy += 0.12`) y decaimiento de opacidad.

El componente soporta un sistema de **propagación entre botones del mismo grupo** (`groupId`): al interactuar con un botón, este emite un `CustomEvent` al documento (`liquid-spread`, `liquid-click`, `liquid-proximity`) que es capturado por todos los botones del mismo grupo. Los botones próximos a la fuente del evento reaccionan con ondulaciones de intensidad proporcional a la distancia, creando la sensación de que el líquido se propaga físicamente entre botones adyacentes.

El color activo del botón es configurable mediante la prop `activeColor` (blue, red, yellow, purple, green, teal, orange), lo que permite usar el mismo componente para acciones semánticamente distintas manteniendo coherencia visual.

4.5.5 Componente propio: StarRating

`StarRating.jsx` gestiona la valoración personal del usuario en Trakt (escala 1–10). Utiliza `LiquidButton` como *trigger* con `activeColor="yellow"`: si el usuario ya tiene una valoración, el botón muestra el valor numérico con el efecto de partículas activo; si no, muestra el icono de estrella.

Al hacer clic, el componente abre un modal renderizado mediante `createPortal()` directamente en `document.body`, evitando problemas de apilamiento CSS (`z-index`) con los elementos de la página de detalle. El modal contiene:

- Un número grande que muestra el valor actual centrado sobre una estrella decorativa semitransparente.
- Un *slider* HTML nativo (`<input type="range">`) con paso de 0,5 y recorrido de 1 a 10, cuyo *thumb* nativo se oculta con `opacity: 0` y se sustituye por un *thumb* personalizado posicionado con CSS (`left: calc(...)`).
- Botones de ajuste fino $-0{,}5$ / $+0{,}5$ a ambos lados del *slider*.
- Botón de confirmación y, si ya existe una valoración, botón de borrado con icono de papelera.

4.5.6 Componente propio: FavoriteWatchlistButtons

`FavoriteWatchlistButtons.jsx` encapsula los botones de Favorito y Pendiente de la página de detalle. Gestiona el estado de forma optimista: al pulsar el botón, el estado visual cambia inmediatamente (sin esperar la respuesta del servidor), y si la petición a TMDb falla, el estado se revierte y se muestra un mensaje de error. Este patrón elimina la latencia perceptible (normalmente de 300–600 ms) entre la acción del usuario y la respuesta de la interfaz.

Los iconos de los botones cambian según el estado (Heart / HeartOff, BookmarkPlus / BookmarkMinus), siguiendo el principio de retroalimentación directa. Cuando hay una operación en curso, los iconos se sustituyen por un spinner (`Loader2 animate-spin`) de Lucide React.

4.5.7 Sistema de animaciones con Framer Motion

Framer Motion (Framer, 2024) gestiona las animaciones de mayor complejidad del proyecto a través de cuatro patrones principales:

1. Animaciones de entrada escalonada (`staggerChildren`). En el dashboard y en las páginas de listas, las tarjetas no aparecen todas simultáneamente. La variante del contenedor define un

`staggerChildren` de 0,05 s (reducido a 0,02 s para listas largas de más de 30 ítems), de forma que cada tarjeta entra con un ligero retraso respecto a la anterior:

```
const containerVariants = {
  hidden: { opacity: 0 },
  visible: {
    opacity: 1,
    transition: { staggerChildren: 0.05 }
  }
};
const itemVariants = {
  hidden: { opacity: 0, y: 20 },
  visible: { opacity: 1, y: 0 }
};
```

2. `AnimatePresence` para salidas animadas. El sistema de tres vistas (Grid, List, Compact) y las pestañas de la página de detalle utilizan `AnimatePresence` para animar tanto la entrada como la salida de los elementos del DOM. Cuando el usuario cambia de vista, el contenido saliente realiza una animación de salida (`exit`) antes de ser eliminado del DOM, evitando cortes bruscos:

```
<AnimatePresence mode="wait">
  <motion.div
    key={currentView}
    initial={{ opacity: 0, x: 20 }}
    animate={{ opacity: 1, x: 0 }}
    exit={{ opacity: 0, x: -20 }}
    transition={{ duration: 0.2 }}
  />
</AnimatePresence>
```

El modo `"wait"` garantiza que la animación de salida se complete antes de que empiece la de entrada.

3. Drawer y overlays móviles. El menú lateral, la búsqueda en *overlay* móvil y los paneles modales utilizan `motion.div` con valores iniciales fuera del *viewport*: `x: -320` para el drawer (deslizamiento desde la izquierda), `scale: 0.95, opacity: 0` para los modales (apertura con escala). Las transiciones de tipo `tween` con `ease: "easeOut"` producen una deceleración natural al final del movimiento.

4. Carrusel del hero. El carrusel automático de la página principal anima las transiciones entre slides con una combinación de `opacity` (entrada: 0→1, salida: 1→0) y un ligero desplazamiento horizontal (`x: +20→0`). La pausa automática al hacer *hover* se gestiona mediante un `useEffect` que detecta el evento `mouseenter` del contenedor del hero.

Performance de las animaciones. Todas las animaciones de Framer Motion utilizan exclusivamente las propiedades CSS `transform` (para `x`, `y`, `scale`, `rotate`) y `opacity`, que el navegador puede procesar enteramente en la GPU sin necesidad de recalcular el *layout* ni repintar capas del DOM. Esto garantiza que las animaciones se ejecuten a 60 fps incluso en dispositivos de gama media.

4.5.8 Sistema de estilos global (`globals.css`)

El fichero `globals.css` define un conjunto de utilidades y animaciones CSS que complementan a Tailwind CSS:

- **`.no-scrollbar`**: oculta la barra de desplazamiento en los carruseles horizontales y los paneles de pestañas, sin deshabilitar el *scroll* táctil.
- **`.sv-scroll`**: barra de desplazamiento personalizada para los paneles con mucho contenido (lista de episodios, resultados de búsqueda): fina, semitransparente, con bordes redondeados y espacio interior.
- **`.animate-shimmer`**: efecto de carga tipo *skeleton* con un gradiente que avanza horizontalmente (`background-position: -200% → 200%`) sobre los *placeholders* de las tarjetas mientras los datos se cargan.
- **`.hover-lift`**: eleva el elemento 4 px y añade una sombra al hacer *hover*, indicando que es interactivo.
- **`.glass / .glass-dark`**: superficies de *glassmorphism* en variante clara y oscura reutilizables en cualquier componente.

4.5.9 Sistema de tres vistas (Grid, List, Compact)

El sistema de tres vistas es un componente transversal utilizado en Favoritos, Pendientes, Historial y En Progreso. La selección de vista se persiste en `localStorage` bajo una clave específica por módulo (`showverse:view:favorites`, `showverse:view:history`, etc.), de forma que el usuario puede tener configuraciones distintas en cada sección.

La transición entre vistas utiliza `AnimatePresence` de Framer Motion con `mode="wait"` y un efecto de *cross-fade* suave: el contenido saliente desaparece con `opacity: 1→0` antes de que el nuevo contenido entre con `opacity: 0→1`. Esto evita los saltos bruscos de *layout* que se producirían si los dos estados coexistieran en el DOM simultáneamente.

4.6 Rendimiento en producción

Las métricas obtenidas en producción (Vercel) mediante Lighthouse validan las decisiones de arquitectura tomadas:

Métrica	**Valor obtenido**	**Objetivo**
Performance (Lighthouse)	92/100	>90
SEO (Lighthouse)	100/100	100
Best Practices	95/100	>90
LCP	1,8 s	<2,5 s
CLS	0,05	<0,1
FID	45 ms	<100 ms

Tabla 4.5: Métricas Lighthouse en producción

El LCP de 1,8 s se consigue principalmente gracias al SSR: el HTML con el *hero* ya renderizado llega al navegador antes de que se descargue todo el JavaScript. El CLS de 0,05 se logra mediante el uso sistemático de `width` y `height` explícitos en el componente `<Image>` de Next.js (Google Developers, 2024b).

4.7 Sistema de caché multinivel

El rendimiento de *The Show Verse* depende críticamente de la estrategia de caché implementada, que opera en cuatro niveles diferenciados según el tipo de dato y sutasa de cambio.

4.7.1 Nivel 1: Incremental Static Regeneration (ISR)

Las páginas del catálogo (trending, top rated, populares) utilizan ISR con `revalidate` variable según la volatilidad del dato:

- **Catálogo general** (10 minutos): páginas de trending y populares se regeneran cada 600 s. Balance entre frescura y carga del servidor.

- **Detalles de título** (1 hora): los metadatos de películas y series son datos estables que raramente cambian. Revalidación cada 3600 s.
- **Temporal crítico** (no-store): datos personales (favoritos, watchlist, historial) nunca se cachean en servidor, requieren autenticación fresca.

El módulo `lib/api/tmdb.js` aplica `next: {revalidate: TTL}` condicionalmente según el *endpoint* invocado.

4.7.2 Nivel 2: Caché en memoria del proceso Node.js

La API Route `/api/omdb` implementa un Map global persistente durante toda la vida del proceso de Node.js:

```
const omdbCache = new Map();  // { imdbId: { data, expiresAt } }

function getCached(imdbId) {
  const cached = omdbCache.get(imdbId);
  if (cached && Date.now() < cached.expiresAt) {
    return cached.data;
  }
  return null;
}

function setCached(imdbId, data, ttlMs) {
  omdbCache.set(imdbId, {
    data,
    expiresAt: Date.now() + ttlMs
  });
}
```

Las respuestas exitosas se cachean 24 horas; las fallidas (404, timeout), 60 s para reintento rápido. Esta caché persiste entre peticiones HTTP mientras el proceso no se reinicie. En Vercel, esto implica que la caché sobrevive entre invocaciones de la función *serverless* dentro de la misma *warm instance*.

4.7.3 Nivel 3: sessionStorage (cliente)

El componente `FavoritesClient.jsx` y similares implementan caché de puntuaciones OMDb en `sessionStorage` del navegador:

```
const CACHE_KEY = 'omdb_scores';
const TTL = 24 * 60 * 60 * 1000;  // 24 horas

function getCachedScore(imdbId) {
  const raw = sessionStorage.getItem(CACHE_KEY);
  if (!raw) return null;

  const cache = JSON.parse(raw);
  const entry = cache[imdbId];

  if (entry && Date.now() < entry.expiresAt) {
    return entry.data;
  }
  return null;
}
```

La caché de `sessionStorage` persiste durante toda la navegación del usuario (incluyendo recargas de página F5), pero se elimina al cerrar la pestaña. Esto equilibra entre persistencia útil y almacenamiento controlado.

4.7.4 Nivel 4: localStorage (cliente)

Las puntuaciones de IMDb (`imdbRating`), Trakt (`traktRating`) y OMDb (`rtScore`, `mcScore`) para las listas personales (Favoritos, Pendientes) se cachean en `localStorage` con TTL de 30 días:

```
const RATINGS_KEY = 'sv_ratings_cache';

function persistRatings(tmdbId, ratings) {
  const cache = JSON.parse(
    localStorage.getItem(RATINGS_KEY) || '{}'
  );
```

```
  cache[tmdbId] = {
    ...ratings,
    cachedAt: Date.now()
  };

  localStorage.setItem(RATINGS_KEY, JSON.stringify(cache));
}
```

Ventaja sobre `sessionStorage`: la caché persiste entre sesiones del navegador, reduciendo drásticamente el número de peticiones a OMDb y Trakt en visitas recurrentes.

Gestión de tamaño: al superar 500 entradas cacheadas, se aplica un algoritmo LRU (*Least Recently Used*) que elimina automáticamente las entradas más antiguas, garantizando que el almacenamiento no crece indefinidamente.

4.7.5 Deduplicación en vuelo

El módulo `lib/api/omdb.js` implementa deduplicación de peticiones concurrentes:

```
const inflightRequests = new Map();  // { imdbId: Promise }

async function fetchOMDB(imdbId) {
  // Si ya hay fetch en curso, esperar su resultado
  if (inflightRequests.has(imdbId)) {
    return await inflightRequests.get(imdbId);
  }

  // Iniciar nuevo fetch
  const promise = realFetch(imdbId);
  inflightRequests.set(imdbId, promise);

  // Limpiar al completar
  promise.finally(() => {
    inflightRequests.delete(imdbId);
  });

  return await promise;
```

```
}
```

Si 10 tarjetas de la lista de favoritos solicitan simultáneamente el rating de una misma película, solo se realiza 1 petición HTTP a OMDb. Las otras 9 peticiones esperan el resultado de la primera. Resultado: reducción de 10x en peticiones a OMDb durante la carga inicial de listas.

4.7.6 Métricas de efectividad

Las pruebas en producción con listas de 100+ favoritos muestran:

- **Primera visita** (caché vacía): 100 peticiones a OMDb, tiempo total 8–12 s con pool de concurrencia limitado a 4 peticiones simultáneas.
- **Segunda visita** (caché caliente): 0 peticiones a OMDb, tiempo total <300 ms (lectura de `localStorage`).
- **Reducción estimada**: 70–80 % del tráfico API en usuarios recurrentes.

4.8 Middleware y optimización SEO

El archivo `middleware.js` en la raíz del proyecto implementa un *middleware* Edge de Next.js que intercepta todas las peticiones entrantes antes de que lleguen a cualquier Server Component o API Route. Su función principal es la reescritura de URLs para optimización SEO y detección de bots.

4.8.1 Detección de bots y crawlers

Problema: Los bots de redes sociales (FacebookBot, Twitter Bot, WhatsApp) y motores de búsqueda (Googlebot, Bingbot) solicitan las páginas de detalle para generar previsualizaciones (*Open Graph*, *Twitter Cards*). Estas páginas son Client Components con carga asíncrona de datos, por lo que los bots verían un HTML vacío sin metadatos.

Solución: El *middleware* detecta bots mediante inspección del *header* `User-Agent`:

```
const userAgent = req.headers.get('user-agent') || '';
const isBot = /bot|crawler|spider|facebookexternalhit|
               twitterbot|whatsapp/i.test(userAgent);

if (isBot && req.nextUrl.pathname.match(/^\/details\//)) {
```

```
  // Reescribir a ruta SEO optimizada
  const seoPath = req.nextUrl.pathname
    .replace('/details/', '/s/');
  return NextResponse.rewrite(new URL(seoPath, req.url));
}
```

Las rutas `/s/movie/[id]`, `/s/tv/[id]` y `/s/person/[id]` son Server Components puros que renderizan HTML completo con todos los metadatos Open Graph en el servidor. Los bots reciben contenido listo para indexar sin ejecutar JavaScript.

4.8.2 Metadatos dinámicos Open Graph

Los Server Components de las rutas SEO exportan una función `generateMetadata()`:

```
export async function generateMetadata({ params }) {
  const details = await getDetails('movie', params.id);
  const posterUrl = `https://image.tmdb.org/t/p/w500
                      ${details.poster_path}`;

  return {
    title: `${details.title} (${details.release_date.slice(0,4)})
            - The Show Verse`,
    description: details.overview,
    openGraph: {
      title: details.title,
      description: details.overview,
      images: [{ url: posterUrl }],
      type: 'video.movie'
    },
    twitter: {
      card: 'summary_large_image',
      title: details.title,
      description: details.overview,
      images: [posterUrl]
    }
  };
}
```

Next.js inyecta estas etiquetas meta en el `<head>` del HTML antes de enviarlo al bot. Resultado: previsualizaciones ricas en redes sociales sin configuración manual.

4.8.3 Redirección canónica

Los usuarios humanos que acceden directamente a `/s/movie/123` (por ejemplo, desde un enlace compartido) son redirigidos automáticamente a `/details/movie/123` (la ruta con interfaz completa):

```
// En /s/movie/[id]/page.jsx
export default function SEOMoviePage() {
  redirect(`/details/movie/${params.id}`);
}
```

Esta estrategia garantiza que:

- Los bots indexan las rutas SEO con metadatos completos.
- Los usuarios navegan las rutas interactivas con Client Components.
- No hay contenido duplicado (problema SEO): las rutas SEO no se indexan para usuarios humanos.

4.9 Sistema de artwork personalizado

El archivo `artwork-overrides.json` en la raíz del proyecto permite sobrescribir manualmente el *poster* o *backdrop* de cualquier título con una imagen externa o una imagen alternativa de la galería de TMDb.

4.9.1 Motivación

TMDb selecciona automáticamente el *poster* con mayor número de votos para cada región lingüística. Sin embargo, en algunos casos el *poster* automático no es el más representativo:

- El póster en español puede ser visualmente inferior al en inglés.
- Para series antiguas, el póster preferido puede ser el original de su estreno y no el rediseñado recientemente.
- Algunos títulos tienen pósters promocionales con mejor reconocimiento visual que el póster oficial.

4.9.2 Formato del archivo

```
{
  "1399": {
    "poster": "/custom/poster-breaking-bad.jpg",
    "backdrop": "https://external.site/backdrop.jpg"
  },
  "550": {
    "poster": "/rCzpDGLbOoPwLjy3OAm5NUPOTrC.jpg"
  }
}
```

Las claves son TMDb IDs. Los valores pueden ser:

- Ruta relativa al *public folder* (`/custom/...`).
- URL de TMDb (`/rCzpDGLbOoPwLjy3OAm5NUPOTrC.jpg`) que se resuelve automáticamente como `https://image.tmdb.org/t/p/original/....`
- URL externa completa (`https://...`).

4.9.3 Implementación

La función `getArtwork(type, id, kind)` en `lib/artworkApi.js` consulta el JSON de sobrescrituras antes de devolver la imagen:

```
import overrides from '@/artwork-overrides.json';

export function getArtwork(type, tmdbId, kind) {
  const override = overrides[String(tmdbId)]?.[kind];

  if (override) {
    // URL externa completa
    if (override.startsWith('http')) {
      return override;
    }
    // Ruta TMDb
```

```
    if (override.startsWith('/') &&
        !override.startsWith('/custom')) {
      return `https://image.tmdb.org/t/p/original${override}`;
    }
    // Ruta pública local
    return override;
  }

  // Fallback: imagen automática de TMDb
  return fetchDefaultArtwork(type, tmdbId, kind);
}
```

Este patrón permite control manual fino sobre la presentación visual de cada título sin modificar código.

4.9.4 API Route de gestión

La ruta `/api/artwork` permite añadir sobrescrituras dinámicamente mediante POST (requiere autenticación):

```
POST /api/artwork
{
  "tmdbId": "1399",
  "kind": "poster",
  "url": "/custom/new-poster.jpg"
}
```

El endpoint lee el JSON actual, añade la entrada y sobrescribe el archivo. Útil para administración sin editar manualmente el JSON.

4.10 Integración con comunidad Trakt

4.10.1 Sistema de comentarios

La página de detalle incluye una pestaña «Comunidad» que muestra los comentarios públicos de Trakt para el título actual. API Route implementada: `/api/trakt/community/comments`.

Ordenación disponible:

- `likes`: comentarios con más *likes* primero (default).
- `newest`: más recientesprimero.
- `oldest`: históricos primero.
- `replies`: con más respuestas primero.

Paginación: 20 comentarios por página, con carga bajo demanda al hacer *scroll* hasta el final (`IntersectionObserver`).

Renderizado: cada comentario muestra:

- Avatar y username del autor (link a perfil Trakt).
- Texto del comentario (Markdown renderizado con `react-markdown`).
- *Spoiler warning*: los comentarios marcados como *spoiler* en Trakt están ocultos por defecto, requieren clic para expandir.
- Contador de *likes* y respuestas.
- Fecha de publicación relativa («hace 3 días»).

4.10.2 Listas curadas de comunidad

La ruta `/api/trakt/community/lists` obtiene listas públicas populares de Trakt para un título:

```
GET /api/trakt/community/lists?type=movie&tmdbId=550
    &tab=popular&page=1&limit=10
```

Pestañas disponibles:

- `popular`: listas con más *likes*.
- `trending`: listas con actividad reciente.
- `personal`: listas del usuario autenticado que contienen este título.

Cada lista muestra: nombre, descripción, autor, número de ítems, número de *likes*, y botón de acceso rápido para ver la lista completa en `/lists/trakt/[listId]`.

4.10.3 Estadísticas de temporada

Para series, la pestaña «Comunidad» incluye ratings promedio de Trakt por temporada y episodio, obtenidos de `/api/trakt/community/seasons`.

Visualización: gráfico de líneas (`recharts`) mostrando evolución del rating episodio a episodio. Permite detectar patrones como:

- Caída de calidad en temporadas intermedias.
- Episodios destacados (picos de rating).
- Consistencia de calidad a lo largo de la serie.

4.11 Integración con Letterboxd

Letterboxd es la plataforma de referencia para cinéfilos y críticos de cine, con más de 10 millones de usuarios. Aunque no tiene API pública, *The Show Verse* implementa integración mediante scraping controlado.

4.11.1 Búsqueda de películas en Letterboxd

La API Route `/api/links/letterboxd` construye URLs de búsqueda en Letterboxd:

```
GET /api/links/letterboxd?title=Inception&year=2010

Response:
{
  "searchUrl": "https://letterboxd.com/search/Inception/",
  "filmSlug": "inception-2010",
  "directUrl": "https://letterboxd.com/film/inception-2010/"
}
```

Algoritmo de construcción del slug:

1) Normalizar título: minúsculas, eliminar caracteres especiales, espacios → guiones.
2) Añadir año si el título es ambiguo (ej. «Dune» → `dune-2021` vs. `dune-1984`).
3) Validar existencia mediante `HEAD` a la URL construida (código 200 confirma).

Si la URL directa falla, se devuelve la URL de búsqueda como fallback.

4.11.2 Botón de acceso rápido

La página de detalle incluye un botón «Ver en Letterboxd» en la sección de enlaces externos que abre la película directamente en Letterboxd en nueva pestaña. Útil para usuarios que desean:

- Leer reseñas detalladas de la comunidad cinéfila.
- Añadir la película a sus listas personales de Letterboxd.
- Ver las listas curadas que incluyen esta película.

4.12 Manejo de errores y estados de carga

4.12.1 Estrategia de *graceful degradation*

Todas las integraciones con APIs externas implementan un patrón de degradación elegante: si una API falla, la aplicación continúa funcionando con funcionalidad reducida en lugar de mostrar un error total.

Ejemplo: fallos de OMDb.

```
try {
  const omdbData = await fetchOMDB(imdbId);
  setImdbRating(omdbData.imdbRating);
} catch (err) {
  console.warn('OMDb unavailable:', err.message);
  setImdbRating(null);  // Mostrar "N/A" en lugar de error
}
```

La tarjeta de la película muestra «IMDb: N/A» en lugar de romper todo el render.

4.12.2 Skeleton loaders

Durante la carga de datos, los componentes renderizan *skeletons* animados con el patrón `animate-pulse` de Tailwind:

- **PosterSkeleton**: rectángulo con proporción 2:3, fondo gris animado.
- **CardSkeleton**: tarjeta completa con bloques de texto simulados.
- **GridSkeleton**: grid de *skeletons* para listas.

- **ScoreboardSkeleton**: barra con círculos animados para ratings.

Los *skeletons* se definen en `components/details/LoadingSkeleton.jsx` y se reutilizan en todos los módulos.

4.12.3 Mensajes de error contextua les

Los errores críticos (autenticación fallida, timeout de red) muestran notificaciones no intrusivas en la esquina superior derecha mediante el patrón *toast*:

```
function showToast(message, type = 'error') {
  const toast = document.createElement('div');
  toast.className = `toast toast-${type}`;
  toast.textContent = message;
  document.body.appendChild(toast);

  setTimeout(() => toast.remove(), 5000);
}
```

Los *toasts* desaparecen automáticamente tras 5 s, evitando que el usuario tenga que cerrarlos manualmente.

4.12.4 Estados vacíos

Las listas vacías (cero favoritos, cero historial) muestran estados vacíos ilustrados con mensajes motivacionales:

```
{favoritos.length === 0 && (
  <div className="empty-state">
    <HeartIcon className="w-16 h-16 text-gray-600" />
    <p className="text-xl">Aún no tienes favoritos</p>
    <p className="text-sm text-gray-400">
      Explora el catálogo y marca tus películas
      y series favoritas
    </p>
    <Link href="/discover">
      <button>Explorar contenido</button>
```

```
    </Link>
  </div>
)}
```

Incluyen un call-to-action (CTA) que guía al usuario hacia la acción relevante (en este caso, descubrir contenido).

4.13 Diseño responsive y adaptación móvil

4.13.1 Breakpoints y estrategia mobile-first

El diseño utiliza los breakpoints estándar de Tailwind CSS:

Breakpoint	Ancho mínimo	Dispositivos objetivo
`sm`	640 px	Móviles grandes (landscape)
`md`	768 px	Tablets
`lg`	1024 px	Laptops
`xl`	1280 px	Desktops
`2xl`	1536 px	Pantallas grandes

Tabla 4.6: Breakpoints responsive

La estrategia *mobile-first* implica que todos los estilos base se definen para móvil (320 px), y se sobrescriben progresivamente para pantallas más grandes:

```
<div className="grid grid-cols-2 md:grid-cols-4
                lg:grid-cols-6 gap-4">
```

Resultado: 2 columnas en móvil, 4 en tablet, 6 en desktop.

4.13.2 Navegación adaptativa

La Navbar implementa dos arquitecturas completamente diferentes según el dispositivo:

Desktop (lg+):

- Barra horizontal superior con tres zonas (logo, búsqueda, links).
- Búsqueda siempre visible en centro.

- Todos los links principales visibles simultáneamente.
- Dropdown de avatar en esquina superior derecha.

Móvil (<lg):

- Barra superior simplificada: logo centrado + botón menú + botón búsqueda.
- *Tab bar* inferior fijo con 4 secciones principales (Inicio, Descubrir, Favoritos, Perfil).
- Drawer lateral deslizante (`translate-x`) para menú completo.
- Búsqueda en *overlay* a pantalla completa.

La transición entre modos ocurre automáticamente en `1024px` sin recarga de página.

4.13.3 Imágenes responsive

El componente `<Image>` de Next.js genera automáticamente `srcset` con múltiples resoluciones:

```
<Image
  src="/poster.jpg"
  width={300}
  height={450}
  sizes="(max-width: 768px) 50vw,
         (max-width: 1024px) 33vw,
         25vw"
  alt="Poster"
/>
```

El navegador selecciona la resolución óptima según el tamaño de renderizado real. Resultado: ahorro de ancho de banda en móviles (imagen 300 px en lugar de 1000 px).

Capítulo 5
Conclusiones

El trabajo ha demostrado que los objetivos planteados son alcanzables con las tecnologías actuales de desarrollo web y sin necesidad de infraestructura de servidor propia. Los resultados obtenidos permiten extraer las siguientes conclusiones.

En primer lugar, la arquitectura Next.js App Router con React Server Components ha demostrado ser particularmente adecuada para aplicaciones de este tipo, que combinan una gran cantidad de datos externos con la necesidad de interactividad en cliente. La separación entre la obtención de datos (servidor) y la presentación interactiva (cliente) simplifica el flujo de datos y reduce las fuentes de error. Los Server Components permiten realizar *fetching* paralelo de múltiples APIs en el servidor sin coste para el cliente, lo que resulta en tiempos de carga significativamente inferiores a los de una SPA equivalente.

En segundo lugar, la integración de OAuth 2.0 mediante API Routes de Next.js ha probado ser una estrategia robusta y segura sin necesidad de librerías de autenticación genéricas. El patrón de almacenar tokens en cookies `httpOnly` y delegar toda la lógica de autenticación al servidor cumple con las recomendaciones de seguridad del RFC 6749 (Hardt, 2012) y las guías de OWASP (OWASP, 2024) para aplicaciones OAuth. La implementación manual permitió además un control fino sobre el flujo de refresco de tokens y la gestión de errores específicos de Trakt.tv.

En tercer lugar, la estrategia de caché diferenciada por tipo de dato ha demostrado ser eficaz en todas sus capas: ISR en servidor para el catálogo, `no-store` para los datos de usuario protegidos por autenticación, y una caché en cliente de cuatro niveles (memoria del proceso, sessionStorage, localStorage y deduplicación en vuelo) para las puntuaciones externas de IMDb, Trakt y OMDb en las vistas de listas, reduciendo drásticamente el número de llamadas a APIs externas. Las métricas en producción muestran una reducción del 70–80 % en el tráfico API para usuarios recurrentes.

En cuarto lugar, el sistema de integración con Plex Media Server demuestra que es técnicamente viable conectar contenido local del usuario con datos de plataformas en la nube. La implementación del flujo de autenticación JWT con refresco automático de tokens y fallback entre HTTP/HTTPS permite una experiencia fluida incluso en configuraciones de red complejas. Esta funcionalidad

diferencia sustancialmente el proyecto de las plataformas de *tracking* existentes.

En quinto lugar, la implementación del middleware Edge para detección de bots y reescritura de URLs ha demostrado ser una solución eficiente para el problema de SEO en aplicaciones con Client Components. El patrón de rutas duales (`/details/` para usuarios, `/s/` para bots) con redirección automática garantiza metadatos Open Graph completos sin duplicar lógica de negocio.

Finalmente, desde el punto de vista del diseño, el uso de Framer Motion para las animaciones, el sistema de superficies semitransparentes con desenfoque de fondo (*glassmorphism*) y la arquitectura de navegación adaptativa (barra horizontal en desktop, drawer + tab bar en móvil) ha producido una interfaz visualmente diferenciada respecto a las plataformas de *tracking* existentes. Las métricas de rendimiento obtenidas (Performance 92/100, LCP 1,8 s, CLS 0,05) demuestran que es posible construir interfaces ricas visualmente sin comprometer la experiencia de usuario.

5.1 Logros alcanzados

El proyecto ha cumplido todos los objetivos específicos planteados inicialmente y ha añadido funcionalidades adicionales significativas:

Objetivos iniciales cumplidos:

1) ✓ Integración completa con TMDb para metadatos de películas y series.

2) ✓ Sistema de autenticación OAuth 2.0 con Trakt.tv con refresh automático.

3) ✓ Módulo de gestión personal con sincronización bidireccional.

4) ✓ Sistema de seguimiento por episodios y temporadas.

5) ✓ Interfaz responsiva con múltiples modos de visualización.

6) ✓ Sistema de caché optimizado.

7) ✓ Despliegue en producción (Vercel).

Funcionalidades adicionales implementadas:

- Sistema de comentarios y listas de comunidad Trakt.
- Integración con Plex Media Server para biblioteca local.
- Sistema de artwork personalizado con sobrescrituras manuales.
- Middleware de SEO con detección de bots y metadatos Open Graph.

- Integración con Letterboxd para enlaces directos.
- Sistema de caché multinivel (4 capas) con deduplicación en vuelo.
- Componente LiquidButton con efectos Canvas avanzados.
- Grid de ratings episódicos con visualización de calidad por temporada.
- Sistema de tres vistas (Grid, List, Compact) con persistencia de preferencias.
- Calendario de estrenos con filtrado por fecha y región.
- Estadísticas de usuario con gráficos interactivos (Recharts).
- Página de biblioteca unificada con contenido en progreso.
- Sistema de actualización optimista en todas las acciones de usuario.
- Manejo de errores con *graceful degradation* y *skeleton loaders*.
- Diseño responsive con navegación adaptativa (desktop vs. móvil).

5.2 Discusión

El trabajo presenta algunas limitaciones que deben reconocerse.

Ausencia de tests automatizados. El proyecto no incluyó una suite de tests unitarios ni de integración durante el desarrollo. La validación se realizó exclusivamente mediante pruebas manuales. Los componentes con lógica de negocio compleja (cálculo de progreso de episodios, normalización de fechas para Trakt, estrategia de selección de *backdrops*, algoritmo de deduplicación en vuelo) serían candidatos prioritarios para la cobertura de tests unitarios. La integración de Jest y React Testing Library sería el siguiente paso natural del proyecto.

Dependencia de APIs de terceros. La plataforma depende de la continuidad y estabilidad de cinco APIs externas (TMDb, Trakt.tv, OMDb, JustWatch y Plex), ninguna de las cuales está bajo el control del proyecto. Un cambio en los términos de servicio, límites de uso o estructura de datos de cualquiera de estas APIs podría afectar la funcionalidad de la aplicación. Esta es una limitación estructural de la arquitectura elegida, aceptada conscientemente por las ventajas que ofrece frente a la alternativa de construir una base de datos propia. El patrón de *graceful degradation* implementado mitiga parcialmente este riesgo.

Accesibilidad parcial. La puntuación de accesibilidad en Lighthouse (88/100) indica que, aunque se han aplicado prácticas básicas (contraste de color, etiquetas ARIA, textos alternativos en imágenes),

no se ha alcanzado el nivel WCAG 2.1 AA completo (W3C, 2023). En particular, la navegación exclusiva por teclado en los modales de episodios y el sistema de tres vistas requiere trabajo adicional. La implementación de `react-focus-lock` y la auditoría con lectores de pantalla (NVDA, VoiceOver) serían pasos necesarios para alcanzar conformidad total.

Escalabilidad del sistema de caché. El uso de `localStorage` para puntuaciones tiene un límite práctico de 5–10 MB según navegador. Para usuarios con listas de 500+ favoritos, este límite podría alcanzarse. La implementación del algoritmo LRU mitiga el problema pero no lo elimina completamente. Una migración a IndexedDB (límite 50 GB) sería recomendable para escalabilidad a largo plazo.

Integración de JustWatch no oficial. La ausencia de API oficial de JustWatch obligó a utilizar los endpoints GraphQL internos de su aplicación web. Aunque funcionan de forma estable, no hay garantía de continuidad. Cualquier cambio en la estructura de las queries GraphQL requeriría adaptar el cliente.

5.3 Líneas de investigación futuras

A partir del trabajo realizado, se identifican las siguientes líneas de desarrollo futuro:

Implementación de Progressive Web App (PWA). La conversión de la aplicación en una PWA, mediante la adición de un *Service Worker* y un *manifest* web, permitiría la instalación en dispositivos móviles y habilitaría un modo de funcionamiento *offline* básico. El *Service Worker* podría cachear las imágenes de pósters y los datos de catálogo para consulta sin conexión. Next.js 13+ soporta PWA mediante el plugin `next-pwa`.

Sistema de recomendaciones personalizado. El historial de visionado acumulado por el usuario constituye un conjunto de datos con potencial para la implementación de algoritmos de recomendación colaborativa o basada en contenido. TensorFlow.js en el cliente o una función *serverless* con Python + scikit-learn permitirían generar recomendaciones personalizadas complementarias a las de Trakt. La técnica de *matrix factorization* (SVD) o redes neuronales de *collaborative filtering* serían candidatas naturales.

Expansión de las funcionalidades sociales. La comparación del historial entre usuarios, las recomendaciones de amigos y la visualización del perfil público de otros usuarios de Trakt son funcionalidades que la API de Trakt soporta pero no han sido implementadas en la versión actual. Un módulo de «Amigos» con feed de actividad reciente aumentaría significativamente el componente social de la plataforma.

Backend propio y base de datos local. A largo plazo, la eliminación de la dependencia total de Trakt.tv como fuente de verdad requeriría la implementación de un *backend* propio con base de datos

(PostgreSQL o MySQL) que almacenara el historial y las preferencias localmente. Esto permitiría además implementar funcionalidades propias como sesiones de visionado grupal (*watch parties*), sistemas de *achievements* y estadísticas avanzadas no disponibles en Trakt.

Integración con más plataformas. La arquitectura modular del sistema de integración API permitiría añadir fácilmente nuevas fuentes: MyAnimeList para anime, Goodreads para literatura adaptada, Metacritic para puntuaciones adicionales, o incluso Spotify/Apple Music para bandas sonoras. Cada integración seguiría el mismo patrón: API Route de *proxy*, caché multinivel, *graceful degradation*.

Sistema de notificaciones push. Notificaciones web (*Web Push API*) para alertar al usuario de:

- Nuevos episodios de series seguidas.
- Respuestas a comentarios en Trakt.
- Nuevas películas añadidas al catálogo que coinciden con sus gustos.
- Amigos que han visto contenido que el usuario tiene en pendientes.

Análisis avanzado con IA. Integración con modelos de lenguaje (GPT-4, Claude) para:

- Generación de resúmenes personalizados de temporadas.
- Respuestas a preguntas sobre la trama sin spoilers.
- Recomendaciones explicadas («Te recomiendo X porque...»).
- Detección de contenido similar por temas y no solo por género.

Bibliografía

Babich, N. (2021). *Glassmorphism in User Interfaces*. UX Planet. https://uxplanet.org/glassmorphism-in-user-interfaces-1f39bb1308c9

CNMC. (2024). *Informe sobre el consumo de vídeo en España*. https://www.cnmc.es/

Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. *Doctoral Dissertation, University of California*. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

Framer. (2024). *Framer Motion Documentation*. https://www.framer.com/motion/

Google Developers. (2024a). *Core Web Vitals report*. https://support.google.com/webmasters/answer/9205520

Google Developers. (2024b). *Web Vitals — Essential metrics for a healthy site*. https://web.dev/vitals/

Hardt, D. (2012). *RFC 6749: The OAuth 2.0 Authorization Framework*. IETF. https://www.rfc-editor.org/rfc/rfc6749

Letterboxd. (2024). *About Letterboxd*. https://letterboxd.com/about/

Meta Open Source. (2024). *React Documentation — React Server Components*. https://react.dev/reference/rsc/server-components

OWASP. (2024). *OWASP Top 10: Broken Authentication*. https://owasp.org/www-project-top-ten/

Schwartz, B. (2004). *The Paradox of Choice: Why More Is Less*. Ecco/HarperCollins.

Statista. (2024). *Number of Netflix subscribers worldwide from 1st quarter 2013 to 4th quarter 2023*. https://www.statista.com/statistics/250934/quarterly-number-of-netflix-streaming-subscribers-worldwide/

The Movie Database. (2024). *TMDb API Documentation v3*. https://developer.themoviedb.org/docs/getting-started

Trakt. (2024). *Trakt API Documentation v2*. https://trakt.docs.apiary.io/

Vercel. (2024). *Next.js Documentation — App Router*. https://nextjs.org/docs/app

W3C. (2023). *Web Content Accessibility Guidelines (WCAG) 2.1*. https://www.w3.org/TR/WCAG21/

Anexo

Variables de entorno del proyecto

La siguiente tabla recoge todas las variables de entorno necesarias para el despliegue de *The Show Verse*.

Variable	**Descripción**	**Requerida**
`NEXT_PUBLIC_TMDB_API_KEY`	API Key de TMDb v3	Sí
`TMDB_SESSION_ID`	Session ID para operaciones de cuenta TMDb	Cuentas
`NEXT_PUBLIC_TRAKT_CLIENT_ID`	Client ID de la aplicación Trakt	Sí
`TRAKT_CLIENT_SECRET`	Client Secret de Trakt (solo servidor)	Sí
`TRAKT_REDIRECT_URI`	URI de callback OAuth registrado en Trakt	Sí
`OMDB_API_KEY`	API Key de OMDb	Para IMDb/RT
`PLEX_URL`	URL del servidor Plex	Para Plex
`PLEX_URLS`	URLs adicionales Plex (separadas por coma)	Opcional
`PLEX_TOKEN`	Token de autenticación Plex (modo simple)	Modo simple
`PLEX_JWT_PRIVATE_KEY`	Clave privada RSA/Ed25519 (modo JWT)	Modo JWT
`PLEX_JWT_KID`	Key ID del JWK (modo JWT)	Modo JWT
`PLEX_CLIENT_IDENTIFIER`	Identificador único de la app en Plex	Modo JWT
`PLEX_MACHINE_IDENTIFIER`	Machine ID del servidor Plex (fallback)	Opcional

Tabla 5.1: Variables de entorno del proyecto

Resumen del sistema de caché multicapa

Nivel	**Almacenamiento**	**TTL**
1 – ISR servidor	Data Cache de Next.js	10 min / 1 h
2 – CDN Vercel	Edge Network	1 – 24 h
3 – Memoria proceso	`Map` en `globalThis`	24 h (éxito) / 60 s (fallo)
4 – `sessionStorage`	Navegador (sesión)	24 h
5 – `localStorage`	Navegador (persistente)	30 días
6 – `Map` en módulo	Memoria del cliente (tab)	10 min / 45 s

Tabla 5.2: Sistema de caché multicapa